Spring Boot + Core Java Interview Bootcamp

5-Day Crash Course for the Java Microservices Developer Interview
Complete Master Edition — Modules 1–11

Targeting interviews at Google · Amazon · Microsoft · Uber · Stripe · Netflix · Oracle · VMware · TCS · Infosys · Cognizant · Accenture · Capgemini · IBM · Deloitte · EY · Wipro · LTIMindtree

Every topic: why it's asked · internals · ASCII memory diagrams · production examples · traps · best answers · code · follow-ups · cheat sheets

Generated July 03, 2026

Contents

Module 1 — Core Java

> Highest priority. Rounds 1 and 2 at every service company (TCS, Infosys, Cognizant, Accenture, Capgemini, Wipro, LTIMindtree) and the "Java fundamentals" screen at product companies live here. If you are shaky on JVM memory, OOP, and `String`, you will not pass.

Node.js bridge: You already know V8, the event loop, and `require`. Java's JVM is V8's much older, heavier cousin: explicit typing, ahead-of-time byte-code compilation, real OS threads (not a single event loop), and a garbage collector you can actually tune.

1.1 JVM, JDK, JRE & the Compilation Process

1. Why Interviewers Ask This

It's the fastest way to tell a "Java developer" from someone who copied Spring code. If you can't cleanly separate JDK/JRE/JVM you'll be filtered in the first 5 minutes.

2. Core Concept

- **JDK** (Java Development Kit) = JRE + development tools (`javac`, `jar`, `javadoc`, `jdb`). You need it to *build*.
- **JRE** (Java Runtime Environment) = JVM + core libraries (`java.lang`, `java.util`…). You need it to *run*.
- **JVM** (Java Virtual Machine) = the abstract machine that executes byte-code. It's a *specification*; HotSpot is Oracle's implementation.

```
JDK  ⊇  JRE  ⊇  JVM
(build)  (run)   (execute bytecode)
```

3. Internal Working — Compilation & Execution

```
Person.java  --javac-->  Person.class (bytecode)  --> JVM ClassLoader
                                                        |
                                              +---------+---------+
                                              | Bytecode Verifier |
                                              +---------+---------+
                                                        |
                                       Interpreter (fast start) + JIT compiler
                                                        |
                                              native machine code (cached)
```

- `javac` compiles source to **platform-independent byte-code** (`.class`).
- JVM **interprets** byte-code initially (fast startup).
- The **JIT (Just-In-Time) compiler** watches for **hot** methods (default threshold ~10,000 invocations) and compiles them to native code. HotSpot has two JITs: **C1** (client, quick) and **C2** (server, aggressive optimizations); modern JVMs use **tiered compilation** (C1 first, then C2).
- "Write once, run anywhere" = byte-code is portable; only the JVM is platform-specific.

4. Memory Diagram

```
+---------------------------- JVM Process ----------------------------+
|  ClassLoader Subsystem  ->  Runtime Data Areas  ->  Execution Engine |
|                              (Heap, Stacks,          (Interpreter,   |
|                               Metaspace, PC,          JIT, GC)       |
|                               Native Method Stack)                   |
+---------------------------------------------------------------------+
```

5. Real Production Example

On a Spring Boot service you ship a fat JAR. Build server needs the **JDK**; the container base image (`eclipse-temurin:21-jre`) only needs the **JRE** — smaller, fewer CVEs. JIT "warm-up" is why the *first* few requests after deploy are slow and p99 latency spikes right after a rolling restart.

6. Most Asked Interview Questions

- Difference between JDK, JRE, JVM? *(follow-up: which do you put in a Docker prod image?)*
- Is Java compiled or interpreted? *(both — bytecode compiled, then interpreted + JIT-compiled)*
- What is JIT? What is tiered compilation? *(follow-up: what is warm-up?)*
- Why is Java platform independent but the JVM is not?

7. Interview Traps

- Saying "Java is purely interpreted" (wrong — JIT compiles to native).
- Saying byte-code is machine code (it's not — it's JVM instructions).
- Thinking the JRE can compile code (it can't; no `javac`).

8. Best Answer

> "`javac` compiles `.java` into portable byte-code. At runtime the JVM's class loader loads and verifies it, the interpreter runs it immediately for fast startup, and the JIT compiler promotes hot methods to native code using tiered C1/C2 compilation. The JDK is what I build with; production containers only ship a JRE to stay small and secure."

9. Coding Example

```
// javac Hello.java  -> Hello.class ; java Hello
public class Hello {
    public static void main(String[] args) {
        System.out.println("Bytecode ran on " + System.getProperty("java.version"));
    }
}
// Inspect bytecode:  javap -c Hello
```

10. Follow-up Coding Questions

- Run `javap -c` on a class and explain a few opcodes (`iload`, `invokevirtual`).
- How would you force C2-only or disable tiered compilation? (`-XX:-TieredCompilation`)

11. Summary

JDK builds, JRE runs, JVM executes. Byte-code is portable; JIT makes it fast.

12. Cheat Sheet

Term	Contains	Purpose
JDK	JRE + javac/jar/javadoc	Development/build
JRE	JVM + libraries	Run apps
JVM	Class loader + memory + engine	Execute bytecode
JIT	C1 + C2 (tiered)	Bytecode → native

1.2 Class Loading

1. Why Interviewers Ask This

`ClassNotFoundException` vs `NoClassDefFoundError`, and how Spring/Tomcat isolate apps, all come from class loading. Frequent at product companies and any “we had a jar hell bug” story.

2. Core Concept

Classes are loaded lazily, on first active use, by a hierarchy of class loaders using **delegation**.

3. Internal Working — Loading → Linking → Initialization

```
1. Loading         : find .class bytes, create Class object in Metaspace
2. Linking
   a. Verification  : bytecode is valid & safe
   b. Preparation   : static fields get default values (0/null/false)
   c. Resolution    : symbolic refs -> direct refs
3. Initialization  : run static blocks & static initializers (assign real values)
```

Parent-delegation model: a loader asks its *parent* first; only if the parent can’t find the class does the child try. Prevents core classes from being spoofed.

```
Bootstrap (rt/core, native)  -> loads java.*  (parent of all)
   ^
Platform/Extension loader    -> loads platform modules
   ^
Application/System loader    -> loads your classpath classes
   ^
(Custom loaders: web apps, plugins, OSGi)
```

4. Memory Diagram

```
new Foo()  ->  AppClassLoader.loadClass("Foo")
                   -> ask Platform loader -> ask Bootstrap loader
                   Bootstrap: "not mine"  Platform: "not mine"
                   App loader loads Foo.class -> Metaspace [Class<Foo>]
```

5. Real Production Example

Two web apps in one Tomcat each get their own `WebAppClassLoader`, so both can use different Jackson versions without clashing. `NoClassDefFoundError` in prod usually = the class was present at compile time but a dependency/jar is missing at runtime (classpath mismatch).

6. Most Asked Interview Questions

- Explain the class loading process. *(3 phases)*
- What is parent delegation and why? *(security + no duplicates)*
- `ClassNotFoundException` vs `NoClassDefFoundError`? *(former: `Class.forName` can’t find at runtime; latter: was there at compile, gone/failed init at runtime)*
- When does a static block run? *(at initialization, once, thread-safely)*

7. Interview Traps

- Confusing the two class errors (very common).
- Thinking loading a class also creates instances (it doesn’t).
- Saying static fields get real values in *preparation* (no — defaults there, real values in *initialization*).

8. Best Answer

> "Loading finds the bytes and builds the `Class` object; linking verifies, gives static fields defaults, and resolves references; initialization runs static initializers once. Loaders delegate to their parent first so core `java.*` classes can't be overridden. That isolation is how one Tomcat runs two apps with conflicting library versions."

9. Coding Example

```
public class Loaders {
    static { System.out.println("static init runs once, at initialization"); }
    public static void main(String[] a) {
        System.out.println(String.class.getClassLoader());       // null = Bootstrap
        System.out.println(Loaders.class.getClassLoader());      // AppClassLoader
    }
}
```

10. Follow-up Coding Questions

- Write a custom `ClassLoader` that loads a class from a byte array.
- Demonstrate lazy loading: prove a class isn't initialized until first active use.

11. Summary

Load → Link (verify, prepare, resolve) → Initialize. Parents get first refusal.

12. Cheat Sheet

Error	When	Cause
`ClassNotFoundException`	reflection/`Class.forName`	name not on classpath
`NoClassDefFoundError`	linking/init	present at compile, missing/failed at runtime

---

1.3 Java Memory Model — Heap, Stack, Metaspace

1. Why Interviewers Ask This

Memory questions separate juniors from seniors and lead straight into GC and `OutOfMemoryError` debugging — a top production topic.

2. Core Concept

- **Heap** — shared, GC-managed; all objects & arrays live here.
- **Stack** — per-thread; holds frames with local variables and references (LIFO).
- **Metaspace** — off-heap (native memory since Java 8); class metadata, replaced PermGen.
- **PC register** & **Native method stack** — per thread, small.

3. Internal Working

- Each thread gets its own **stack**; each method call pushes a **frame** (locals, operand stack, return address). Frame pops on return. Deep recursion → `StackOverflowError`.
- **Heap** is split for generational GC: **Young** (Eden + two Survivor spaces) and **Old/Tenured**.
- **Metaspace** grows in native memory (bounded by `-XX:MaxMetaspaceOnly` if set); before Java 8 this was **PermGen** inside the heap and caused frequent `OutOfMemoryError: PermGen space`.
- **Primitives declared as locals** live on the stack; **object fields** (even primitive fields) live on the heap inside the object.

4. Memory Diagram

```
              JVM MEMORY
+----------------------- HEAP (shared) ------------------------+
|  Young Generation                Old Generation             |
|  +--------+ +----+ +----+         +----------------------+    |
|  | Eden    | | S0 | | S1 |         |  long-lived objects  |    |
|  +--------+ +----+ +----+         +----------------------+    |
+---------------------------------------------------------------+
+-- Thread-1 Stack --+  +-- Thread-2 Stack --+   +- Metaspace -+
| frame: main()      |  | frame: run()       |   | Class meta  |
|  int x=5 (value)   |  |  ref o ----------------->[Object]    |
|  ref p ----------------------------------->[Object on heap]|
+--------------------+  +--------------------+   +-------------+
```

5. Real Production Example

A memory leak in a Spring app = objects unintentionally reachable (e.g., a static `Map` cache never evicted) → Old gen fills → `OutOfMemoryError: Java heap space`. You capture a heap dump (`-XX:+HeapDumpOnOutOfMemoryError`) and analyze in Eclipse MAT. `-Xmx2g -Xms2g` fixes heap size; classloader leaks (redeploys) show as Metaspace growth.

6. Most Asked Interview Questions

- Stack vs heap? Where do primitives/objects/references live? *(follow-up: are String literals on stack? No — heap string pool)*
- What replaced PermGen and why? *(Metaspace, native memory, auto-grows)*
- What causes `StackOverflowError` vs `OutOfMemoryError`?
- Is memory allocation thread-safe on the heap? *(yes; TLABs make it fast)*

7. Interview Traps

- “Objects can live on the stack” — no (ignoring escape-analysis scalar replacement, which is an optimization detail).
- “Primitives always on stack” — only *local* primitives; primitive *fields* are on the heap in the object.
- Confusing Metaspace (native) with heap.

8. Best Answer

> “Objects live on the shared heap, split into young and old generations for GC. Each thread has its own stack of frames holding locals and references — references point into the heap. Class metadata moved from PermGen to native Metaspace in Java 8, which auto-grows so we stopped seeing PermGen OOMs. Heap OOM means a leak or undersized `-Xmx`; StackOverflow means unbounded recursion.”

9. Coding Example

```java
public class MemoryDemo {
    int fieldOnHeap = 10;                 // lives in the object on the heap
    void method() {
        int localOnStack = 5;             // stack
        MemoryDemo ref = new MemoryDemo();// 'ref' on stack, object on heap
    }
    static int recurse(int n){ return recurse(n+1); } // -> StackOverflowError
}
```

10. Follow-up Coding Questions

- Trigger and catch a `StackOverflowError`; why can you catch an `Error` but shouldn’t rely on it?
- Write code that causes a heap OOM with an ever-growing `List`.

11. Summary

Heap = objects (shared, GC). Stack = per-thread frames. Metaspace = class metadata (native).

12. Cheat Sheet

Area	Scope	Stores	Error
Heap	shared	objects/arrays	`OutOfMemoryError: Java heap space`
Stack	per-thread	frames, locals, refs	`StackOverflowError`
Metaspace	shared (native)	class metadata	`OutOfMemoryError: Metaspace`

1.4 Garbage Collection, Generational GC & Object Lifecycle

1. Why Interviewers Ask This

GC is *the* differentiator for backend Java. Everyone from TCS to Netflix asks "how does GC work" and "which collector do you use and why".

2. Core Concept

GC automatically reclaims unreachable objects. **Reachability** (from GC roots), not reference counting, decides liveness. Generational GC exploits the **weak generational hypothesis**: *most objects die young.*

3. Internal Working — Generational GC

```
1. New objects  -> Eden.
2. Eden full    -> Minor GC: live objects copied to a Survivor space; dead ones dropped.
3. Survivors aged each Minor GC. After threshold (MaxTenuringThreshold) -> promoted to Old gen.
4. Old gen full -> Major/Full GC (slower, may 'stop the world').
```

- **GC Roots**: stack locals, static fields, active threads, JNI refs. Anything reachable from a root is live.
- **Mark-Sweep-Compact**: mark live, sweep dead, compact to remove fragmentation.
- **Collectors**:
- **Serial** — single thread, tiny heaps.
- **Parallel (Throughput)** — multi-thread, batch jobs.
- **G1 (default since Java 9)** — region-based, predictable pause targets (`-XX:MaxGCPauseMillis`).
- **ZGC / Shenandoah** — sub-millisecond pauses, huge heaps (low-latency services).
- **Stop-The-World (STW)**: application threads pause during certain GC phases. Minimizing STW is the whole game.

4. Memory Diagram

```
Allocation:  [ Eden ] fills -> Minor GC -> survivors -> [ S0/S1 ] -> age -> [ Old ]
GC roots ---> reachable objects kept; unreachable islands (even if they reference
             each other) are collected. Cyclic refs are NOT a leak in Java.
```

5. Real Production Example

A latency-sensitive payment service on a 8GB heap uses **G1** with `-XX:MaxGCPauseMillis=200`. You watch GC logs (`-Xlog:gc*`) for frequent Full GCs (a red flag for leaks or undersized old gen). Netflix-style low-latency services move to **ZGC** to keep pauses under 1ms.

6. Most Asked Interview Questions

- How does GC decide what to collect? *(reachability from GC roots, not ref counting)*
- Explain generational GC / why young & old? *(most objects die young → cheap minor GC)*
- Can you force GC? *(`System.gc()` is only a hint; never rely on it)*
- Difference between Minor, Major, Full GC? What is Stop-The-World?
- Which collectors do you know? When G1 vs ZGC? *(follow-up: does GC prevent all memory leaks? No.)*
- Do circular references cause leaks in Java? *(No — reachability handles cycles.)*

7. Interview Traps

- Saying Java uses reference counting (it doesn't — reachability).
- Claiming `System.gc()` guarantees collection.
- Saying "Java can't leak memory" — it can (unbounded caches, listeners, ThreadLocals, classloaders).
- Confusing `finalize()` with a destructor (see next section).

8. Best Answer

> "The GC frees objects unreachable from GC roots — so cycles aren't a problem. It's generational because most objects die young: cheap minor GCs sweep Eden, survivors age and get promoted to old gen, which is collected less often by an expensive major GC. Modern JVMs default to G1, which is region-based and lets me target a max pause time; for ultra-low latency I'd use ZGC. `System.gc()` is only a hint, and GC doesn't fix logical leaks like an unbounded static cache."

9. Coding Example

```
public class GcDemo {
    public static void main(String[] args) {
        Object a = new Object();
        a = null;               // old object now unreachable -> eligible for GC
        System.gc();            // hint only; do NOT rely on it in code
        // Prefer WeakReference/soft caches to let GC reclaim under pressure:
        java.lang.ref.WeakReference<byte[]> cache =
            new java.lang.ref.WeakReference<>(new byte[1024]);
        System.out.println(cache.get() != null);
    }
}
```

10. Follow-up Coding Questions

- Write a class that leaks via a `static List` and explain how to detect it (heap dump/MAT).
- Explain `WeakReference` vs `SoftReference` vs `PhantomReference` with a cache example.

11. Summary

Reachability-based, generational, mostly automatic. G1 default; ZGC for low pause. GC ≠ leak-proof.

12. Cheat Sheet

Collector	Best for	Pause
Serial	tiny/CLI	high
Parallel	throughput/batch	medium
G1 (default)	general services	tunable (~200ms)
ZGC/Shenandoah	low latency, big heap	<1ms

Object lifecycle: Created (`new`) → In use (reachable) → Unreachable → Eligible → (optional `finalize`) → Collected.

1.5 OOP, SOLID & the Four Pillars

1. Why Interviewers Ask This

Every design and code-quality question rests on OOP + SOLID. "Explain SOLID with an example" is nearly guaranteed in service-company and mid-level product interviews.

2. Core Concept — Four Pillars

- **Encapsulation** — bundle state + behavior; hide internals behind methods (private fields, getters/setters, invariants).
- **Abstraction** — expose *what*, hide *how* (interfaces, abstract classes).
- **Inheritance** — reuse via "is-a" (`Dog extends Animal`).
- **Polymorphism** — one interface, many forms: **compile-time** (overloading) & **runtime** (overriding via dynamic dispatch).

3. Internal Working — Runtime Polymorphism

The JVM keeps a **vtable** (virtual method table) per class. An overridden call (`animal.sound()`) is resolved at runtime by the *actual object's* type via `invokevirtual`, not the reference type. Overloading is resolved at *compile time* by argument types via `invokestatic`/`invokevirtual` with a fixed signature.

4. Memory Diagram

```
Animal a = new Dog();       // ref type Animal, object type Dog
a.sound();
   -> invokevirtual -> look up Dog's vtable -> Dog.sound()   (runtime dispatch)
```

5. SOLID (with production framing)

Letter	Principle	One-liner	Spring example
S	Single Responsibility	one reason to change	Controller ≠ Service ≠ Repository
O	Open/Closed	open to extend, closed to modify	add a new `PaymentStrategy` impl, don't edit existing
L	Liskov Substitution	subtypes usable as base type	`Square extends Rectangle` violates it
I	Interface Segregation	many small interfaces > one fat	split `Repository` not one god-interface
D	Dependency Inversion	depend on abstractions	inject `PaymentGateway` interface, not `StripeClient`

6. Most Asked Interview Questions

- Explain the 4 OOP pillars with real examples.
- Explain SOLID; give a violation and the fix. *(follow-up: which SOLID does DI implement? → D)*
- Compile-time vs runtime polymorphism? *(overloading vs overriding)*
- Is Java 100% OOP? *(No — primitives exist.)*

7. Interview Traps

- Confusing abstraction with encapsulation (abstraction = design/hide complexity; encapsulation = data hiding + bundling).
- Saying overloading is runtime polymorphism (it's compile-time).
- Reciting SOLID with no example — interviewers want the *fix*.

8. Best Answer

> "Encapsulation hides state behind methods to protect invariants; abstraction exposes intent through interfaces; inheritance reuses via is-a; polymorphism lets one reference behave as many types — overloading resolved at compile time, overriding at runtime via the vtable. SOLID keeps that flexible: e.g., Dependency Inversion is exactly what Spring DI gives us — my service depends on a `PaymentGateway` interface, and Spring injects Stripe or a mock."

9. Coding Example

```
interface PaymentGateway { boolean charge(long cents); }          // abstraction + D + O

final class StripeGateway implements PaymentGateway {              // encapsulated impl
    public boolean charge(long cents) { /* call Stripe */ return true; }
}

class CheckoutService {                                            // S: only checkout logic
    private final PaymentGateway gateway;                          // D: depends on abstraction
    CheckoutService(PaymentGateway gateway) { this.gateway = gateway; }
    boolean pay(long cents) { return gateway.charge(cents); }
}
```

10. Follow-up Coding Questions

- Refactor an `if/else` on payment type into the Strategy pattern (Open/Closed).
- Show a Liskov violation with `Rectangle`/`Square` and fix it.

11. Summary

4 pillars + SOLID = maintainable code. DI = Dependency Inversion in practice.

12. Cheat Sheet

`S`ingle · `O`pen-closed · `L`iskov · `I`nterface-seg · `D`ependency-inversion. Overload = compile-time; Override = runtime (vtable).

1.6 Interface vs Abstract Class · Overloading vs Overriding

Interface vs Abstract Class

	Interface	Abstract Class
Multiple inheritance	Yes (implement many)	No (extend one)
State/fields	only `public static final` constants	instance fields allowed
Methods	abstract + `default`/`static`/`private` (Java 8+)	abstract + concrete
Constructor	No	Yes
Use when	capability/contract (“can-do”: `Comparable`)	shared base + state (“is-a”)

Trap: Since Java 8, interfaces have `default` methods → the “interfaces can’t have bodies” answer is outdated. The **diamond problem** with two default methods is resolved by forcing the class to override and call `Interface.super.method()`.

Overloading vs Overriding

	Overloading	Overriding
When	compile-time	runtime
Signature	must differ (params)	must match
Return type	can differ	same/covariant
Access	any	can’t reduce visibility
`static`	can overload	can’t override (hidden, not overridden)

Best answer for “can you override a static method?” → No. A static method belongs to the class; redeclaring it in a subclass **hides** it (resolved by reference type, not object type).

1.7 final · finally · finalize · static · this · super

`final`, `finally`, `finalize` (a guaranteed question)

- **`final`** — keyword. `final` variable = constant; `final` method = can't override; `final` class = can't extend (e.g., `String`).
- **`finally`** — block that always runs after `try/catch` (even on return/exception), used for cleanup. Skipped only on `System.exit()` or JVM crash.
- **`finalize()`** — Object method the GC *might* call before reclaiming. **Deprecated since Java 9, removed in later releases.** Unpredictable, can resurrect objects, hurts GC, no guarantee it runs. Use **`try-with-resources`** / `Cleaner` / `AutoCloseable` instead.

> **Best answer:** "`final` is a modifier, `finally` is a cleanup block, `finalize()` is a deprecated GC hook I never use — I use try-with-resources for deterministic cleanup."

`static`

Belongs to the **class**, not instances: shared across all objects, loaded at class initialization. Static methods can't use `this`/`super` or access instance members directly. Static blocks run once at init. Watch for the trap: static fields + concurrency = shared mutable state (needs synchronization).

`this` vs `super`

- `this` — current object (disambiguate fields, chain constructors `this(...)`).
- `super` — parent (call parent constructor `super(...)`, or parent method `super.foo()`).
- `super(...)`/`this(...)` must be the **first** statement in a constructor.

1.8 String Pool · String vs StringBuilder vs StringBuffer · Immutability

1. Why Interviewers Ask This

`String` is the most-used class and the classic immutability/`==` vs `.equals()` trap. Nearly guaranteed.

2 & 3. Core + Internal

- `String` is **immutable** and `final`. Its `char[]`/`byte[]` (compact strings since Java 9) is private and never mutated.
- **String Pool** (a.k.a. intern pool) lives in the **heap** (since Java 7; was PermGen before). String *literals* are interned — identical literals share one object. `new String("x")` creates a *new* heap object (not pooled unless you call `.intern()`).

4. Memory Diagram

```
String a = "hi";              // pool: ["hi"] <- a
String b = "hi";              // b -> same pooled "hi"      a == b  -> true
String c = new String("hi");  // heap: [new "hi"] <- c      a == c  -> false
                              //                             a.equals(c) -> true
String d = c.intern();        // d -> pooled "hi"           a == d  -> true
```

5. Real Production Example

Building a big log/CSV line by `s += token` in a loop creates a new `String` each iteration → O(n^2) garbage. Use **`StringBuilder`** (single mutable buffer). In a multithreaded logger you'd use **`StringBuffer`** (synchronized) — or better, keep the builder thread-local.

Immutability — why `String` is immutable

Security (used in class loading, network, file paths), thread-safety (shareable without locks), safe as **HashMap keys** (cached `hashCode`), and pool sharing. Any "modification" returns a **new** String.

String vs StringBuilder vs StringBuffer

	String	StringBuilder	StringBuffer
Mutable	No	Yes	Yes
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed	slow for concat loops	fastest	slower (locking)
Use	fixed text, keys	single-thread build	rare, shared build

6. Most Asked Interview Questions

- Why is `String` immutable? *(security, thread-safety, hashcode caching, pool)*
- `==` vs `.equals()` for strings? *(reference vs value)*
- `new String("a") == "a"`? *(false; `.equals` true; `.intern()` fixes)*
- `String` vs `StringBuilder` vs `StringBuffer`? When each?
- How many objects does `String s = new String("a")` create? *(up to 2: literal in pool + heap object)*

7. Interview Traps

- Using `==` to compare string content.
- Saying the pool is in PermGen (it moved to heap in Java 7).
- Using `String +=` in loops.

8. Best Answer

> "`String` is immutable and `final` for security, thread-safety, and so it can be pooled and safely cached as a map key. Literals are interned so equal literals share one object, which is why `==` can be true for literals but false against `new String`. For heavy concatenation I use `StringBuilder`; `StringBuffer` only when a buffer is genuinely shared across threads."

9. Coding Example

```
String a = "hi", b = "hi";
System.out.println(a == b);             // true  (pool)
System.out.println(a == new String("hi")); // false (new heap object)
System.out.println(a.equals(new String("hi"))); // true

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) sb.append(i).append(',');  // O(n), no garbage storm
String csv = sb.toString();
```

10. Follow-up Coding Questions

- Reverse a string in place using `StringBuilder.reverse()`; then without it.
- Explain why using a mutable object as a `HashMap` key is dangerous.

11 & 12. Summary + Cheat Sheet

Immutable String → safe & poolable. `==` = reference, `.equals()` = value. Build with `StringBuilder`; synchronize with `StringBuffer` only if shared.

1.9 Immutable Objects · Wrapper Classes · Autoboxing

Building an immutable class (common coding ask)

```
public final class Money {                 // 1. final class (no subclass)
    private final long cents;              // 2. private final fields
    private final String currency;
    public Money(long cents, String currency) {   // 3. set via constructor
        this.cents = cents;
        this.currency = currency;          // 4. defensively copy mutable inputs (none here)
    }
    public long cents() { return cents; }  // 5. getters only, no setters
    public String currency() { return currency; }
}
```

Rules: `final` class, all fields `private final`, no setters, defensive copies of mutable fields (arrays, `Date`, collections) on the way in and out.

Wrapper Classes & Autoboxing

- Wrappers box primitives into objects: `int↔Integer`, `long↔Long`, etc. Needed for generics/collections (`List<Integer>`).
- **Autoboxing** = automatic `int → Integer`; **unboxing** = reverse. Compiler inserts `Integer.valueOf`/`intValue`.
- **`Integer` cache**: `valueOf` caches -128..127, so `Integer a=100, b=100; a==b` is **true**, but `127`… `Integer a=200,b=200; a==b` is **false** (different objects). Always compare wrappers with `.equals()`.
- **NPE trap**: unboxing a `null Integer` into an `int` throws `NullPointerException`.

> **Traps:** `Integer` `==` comparisons; NPE on unboxing null; autoboxing in tight loops (creates garbage — prefer primitives for performance-critical code).

1.10 Enum · var · Records · Sealed Classes

Enum

Type-safe set of constants; each is a singleton. Can have fields, constructors, methods, and per-constant bodies. Great for strategy/state. Use in `switch`; use `EnumMap`/`EnumSet` (very fast). `enum` is implicitly `final` and extends `java.lang.Enum`.

```
enum Status {
    ACTIVE(1), INACTIVE(0);
    private final int code;
    Status(int code){ this.code = code; }
    int code(){ return code; }
}
```

var (Java 10) — local variable type inference

Only for **local** variables with an initializer. Not for fields, params, or return types. Still statically typed — `var list = new ArrayList<String>();` infers `ArrayList<String>`. Don't overuse; keep readability.

Records (Java 16) — basic awareness

Immutable data carriers: auto-generate constructor, `equals`, `hashCode`, `toString`, accessors.

```
public record Point(int x, int y) {}   // that's a full immutable value class
```

Great for DTOs. Can't extend classes; are implicitly `final`. Interviewers love "records vs Lombok `@Value`".

Sealed Classes (Java 17) — basic awareness

Restrict which classes may extend/implement: `sealed interface Shape permits Circle, Square {}`. Enables exhaustive `switch` and controlled hierarchies. Just know the concept and syntax.

Module 1 — Top 25 Interview Questions (with senior answers)

1. **JDK vs JRE vs JVM?** JDK builds (has `javac`), JRE runs (JVM + libs), JVM executes byte-code. Ship a JRE in prod.
2. **Is Java compiled or interpreted?** Both — compiled to byte-code, then interpreted + JIT-compiled to native.
3. **What is JIT / tiered compilation?** Promotes hot methods to native via C1 then C2.
4. **Class loading phases?** Loading → Linking (verify, prepare, resolve) → Initialization.
5. **Parent delegation?** Child asks parent first — prevents core-class spoofing & duplicates.
6. **`ClassNotFoundException` vs `NoClassDefFoundError`?** Reflection-time missing vs runtime-missing after compile.
7. **Stack vs Heap?** Per-thread frames/locals vs shared objects (GC-managed).
8. **What replaced PermGen?** Metaspace (native memory, auto-grows) in Java 8.
9. **How does GC decide liveness?** Reachability from GC roots, not reference counting.
10. **Generational GC?** Most objects die young → cheap minor GC of Eden; survivors promoted to old gen.
11. **Default collector? Low-latency choice?** G1 default; ZGC/Shenandoah for sub-ms pauses.
12. **Can `System.gc()` force GC?** No — only a hint.
13. **Do cycles leak in Java?** No — reachability handles cycles. Leaks come from unbounded reachable structures.
14. **`final` vs `finally` vs `finalize`?** Modifier vs cleanup block vs deprecated GC hook.
15. **Why is `finalize` deprecated?** Unpredictable, no guarantee, hurts GC — use try-with-resources/Cleaner.
16. **4 OOP pillars?** Encapsulation, Abstraction, Inheritance, Polymorphism.
17. **Explain SOLID + a fix.** e.g., Dependency Inversion → inject an interface, not a concrete client.
18. **Overloading vs overriding?** Compile-time (params) vs runtime (vtable dispatch).
19. **Can you override static/private/final?** No to all — static is hidden; private/final aren't inherited/overridable.
20. **Interface vs abstract class?** Multiple inheritance + contract vs single inheritance + shared state.
21. **Why is String immutable?** Security, thread-safety, hashcode caching, pooling.
22. **`==` vs `.equals()`?** Reference identity vs logical equality.
23. **String vs StringBuilder vs StringBuffer?** Immutable vs mutable-unsynchronized vs mutable-synchronized.
24. **Integer caching / autoboxing traps?** -128..127 cached; compare wrappers with `.equals()`; null unboxing NPE.
25. **What are records/sealed classes for?** Immutable data carriers; restricted hierarchies for exhaustive matching.

Module 1 — Top Coding Questions

- Build an immutable `Money`/`Employee` class (with defensive copies).
- Implement `equals()` and `hashCode()` correctly (and explain the contract).
- Reverse a string; check palindrome; count char frequency with a `Map`.
- Demonstrate the `Integer` cache boundary (`127` vs `128`).
- Write a class hierarchy showing runtime polymorphism, then refactor an `if/else` type-switch into Strategy.

Module 1 — Common Follow-ups

- "Show me the `equals/hashCode` contract." (equal objects → equal hash; used by HashMap.)
- "How would you debug an OOM in prod?" (heap dump + MAT, GC logs, check caches/ThreadLocals.)
- "Why not just use Lombok instead of records?" (records are language-level, immutable, no build-time processor.)

Module 1 — One-Page Cheat Sheet

```
JDK⊇JRE⊇JVM | javac->bytecode->JIT(C1/C2 tiered)
ClassLoad: Load->Verify/Prepare/Resolve->Init | parent-delegation
Heap=objects(GC) Stack=frames(per-thread) Metaspace=class meta(native)
GC: reachability from roots; generational (Eden->Survivor->Old); G1 default, ZGC low-latency
System.gc()=hint. Cycles don't leak. finalize=deprecated -> try-with-resources
OOP: Encapsulation/Abstraction/Inheritance/Polymorphism
SOLID: S O L I D; DI = Dependency Inversion
Overload=compile-time(params); Override=runtime(vtable). static/private/final NOT overridable
Interface: multi-inherit + default methods; Abstract: single + state
String immutable+pooled; ==ref vs .equals value; StringBuilder(fast) StringBuffer(sync)
Integer cache -128..127; compare wrappers with equals; null unbox=NPE
record=immutable DTO; sealed=restricted hierarchy; var=local inference
```

Module 1 — Mock Interview (answer these, then continue)

1. “Walk me through what happens from `javac Foo.java` to the method running as native code.”
2. “My Spring service throws `OutOfMemoryError: Java heap space` at 3 a.m. under load. Walk me through diagnosis and fixes.”
3. “Why can two web apps in one Tomcat use different Jackson versions?”
4. “Give me a real SOLID violation you fixed and how DI helped.”
5. “`Integer a = 128, b = 128; a == b` — true or false, and why? What about `127`?”
6. “Write an immutable `Money` class and justify every keyword.”

Model answers are embedded in the sections above. When ready, continue to Module 2.

Module 2 — Collections Framework

> Highest priority and the single most-asked coding-adjacent topic. “How does HashMap work internally?” is asked in *almost every* Java interview at TCS/Infosys/Cognizant and product companies alike. Know the internals cold.

Node.js bridge: JS has `Array`, `Map`, `Set`, `Object`. Java gives you a rich typed hierarchy with explicit complexity guarantees and thread-safe variants. `HashMap` ≈ JS `Map` but with load factor, buckets, and treeification you can be quizzed on.

2.1 Collection Hierarchy

Core Concept

```
                 Iterable
                    |
                Collection
     +-------------+-------------+
    List           Set          Queue
     |              |              |
ArrayList     HashSet        LinkedList (also List)
LinkedList    LinkedHashSet  PriorityQueue
Vector        TreeSet(NavigableSet)  ArrayDeque (also Deque)
(Stack)
                   Map   (NOT a Collection)
     +-------------+-------------+
  HashMap     LinkedHashMap   TreeMap(NavigableMap)
  Hashtable   ConcurrentHashMap
```

- **`Map` is not a `Collection`** — it's a separate root (key→value). Common trap.
- **List** = ordered, indexed, duplicates allowed.
- **Set** = no duplicates.
- **Queue/Deque** = FIFO / double-ended.

Interfaces vs implementations (senior framing)

Program to the interface: `List<X> l = new ArrayList<>();`, `Map<K,V> m = new HashMap<>();`. Choose the implementation for the access pattern and thread model.

Cheat Sheet — pick the right one

Need	Use
Indexed, mostly reads	`ArrayList`
Frequent add/remove at ends, queue/deque	`ArrayDeque` (not `LinkedList`)
Unique, fast lookup	`HashSet`
Unique + insertion order	`LinkedHashSet`
Unique + sorted	`TreeSet`
Key→value fast	`HashMap`
Key→value + order	`LinkedHashMap` (also LRU)
Key→value + sorted	`TreeMap`
Concurrent map	`ConcurrentHashMap`
Priority ordering	`PriorityQueue`

2.2 ArrayList — Dynamic Array, Resize, Complexity

1. Why Interviewers Ask This

It's the default `List` and the base for "what's the complexity of `add`" and "how does it grow" questions.

2. Core Concept

Resizable array backed by `Object[] elementData`. Random access by index is O(1).

3. Internal Working — growth

- Default capacity **10** (lazily allocated on first add).
- When full, it grows by **~50%**: `newCap = oldCap + (oldCap >> 1)`. (`ArrayList`; `Vector` doubles.)
- Growth = allocate new array + `System.arraycopy` old → new. That single add is O(n), but **amortized O(1)** across many adds.
- `add(index, e)` and `remove(index)` shift elements → O(n).
- Not thread-safe.

4. Memory Diagram

```
size=3, capacity=4:  elementData -> [ A | B | C | _ ]
add(D): full -> newcap = 4 + 2 = 6, copy:
                     [ A | B | C | D | _ | _ ]
```

5. Real Production Example

Loading 1M rows from DB into `new ArrayList<>()` triggers ~20 resizes/copies. Pre-size with `new ArrayList<>(1_000_000)` to avoid the copy churn — a real latency/GC win.

6. Most Asked Questions

- How does ArrayList grow? *(1.5x, arraycopy)*
- Complexity of get/add/add-at-index/remove? *(O(1)/amortized O(1)/O(n)/O(n))*
- ArrayList vs Array? *(dynamic vs fixed, generics, autobox overhead)*
- ArrayList vs LinkedList? (see below)
- Is ArrayList thread-safe? How to make it? *(`Collections.synchronizedList` / `CopyOnWriteArrayList`)*

7. Traps

- Saying it doubles (that's `Vector`; ArrayList is 1.5x).
- Removing while iterating with a for-each → `ConcurrentModificationException` (use `Iterator.remove()`).

8. Best Answer

> "`ArrayList` is a dynamic array; get is O(1). When capacity is exceeded it grows by 50% and arraycopies, so add is amortized O(1) but inserts/removes in the middle are O(n) due to shifting. I pre-size it when I know the count to avoid resize churn."

9. Coding Example

```
List<String> list = new ArrayList<>(1000);       // pre-sized
list.add("a");
// Safe removal during iteration:
Iterator<String> it = list.iterator();
while (it.hasNext()) { if (it.next().isEmpty()) it.remove(); }
```

10. Follow-ups

- Remove all even numbers safely (Iterator vs `removeIf`).

- Convert array↔list (`Arrays.asList` gotcha: fixed-size, backed by array).

11 & 12. Summary + Cheat

Dynamic array, 1.5x growth, O(1) get, O(n) middle ops. Pre-size when possible.

2.3 LinkedList

Doubly-linked list; implements `List` **and** `Deque`. - get(index) = O(n) (walks nodes); add/remove at ends = O(1); add/remove in middle = O(1) *if you hold the node* but O(n) to find it. - Higher memory per element (two pointers + object header). - **In practice, prefer `ArrayList`** for lists and **`ArrayDeque`** for queue/stack — `LinkedList` is rarely the right answer despite the textbook “fast inserts”.

Best answer trap: “Use LinkedList for frequent insertions” — only true when you already have the node reference; otherwise finding the position is O(n). CPU cache locality makes `ArrayList` win in most real workloads.

2.4 HashMap — the flagship internals question

1. Why Interviewers Ask This

The most-asked Java question, period. It tests hashing, buckets, collisions, equals/hashCode, resize, and (post-Java 8) treeification.

2. Core Concept

Stores key→value in an **array of buckets**. The key’s `hashCode()` decides the bucket; `equals()` resolves within a bucket. Average O(1) get/put.

3. Internal Working (Java 8+)

1. `hash(key)` = `h = key.hashCode(); h ^ (h >>> 16)` — spreads high bits into low bits so poorly-distributed hashcodes still scatter.
2. **Bucket index** = `(n - 1) & hash` where `n` = table length (always a power of 2, so `&` = fast modulo).
3. Each bucket is a **`Node` linked list**; on collision, append (Java 8 appends at tail).
4. **Put**: if key’s hash+`equals` matches an existing node → replace value; else add node.
5. **Load factor** default **0.75**. When `size > capacity * loadFactor`, **resize**: capacity doubles (16→32→…), and every node is **rehashed** into the new table.
6. **Treeification**: if a single bucket’s list length ≥ **8** *and* table capacity ≥ **64**, the list converts to a **red-black tree** → worst case O(log n) instead of O(n). If it shrinks below **6**, it **untreeifies** back to a list.
7. Default initial capacity **16**. Null key allowed (stored in bucket 0); null values allowed.

4. Memory Diagram

```
capacity=16, loadFactor=0.75 -> resize at size 12
table:
 [0] -> null
 [1] -> (k1,v1) -> (k9,v9)          <- collision: linked list in bucket 1
 [2] -> null
 ...
 [5] -> (k5,v5) -> ... (>=8 & cap>=64) -> converts to Red-Black TREE

index = (n-1) & hash(key)      hash(key)=h ^ (h>>>16)
```

5. Real Production Example

Using a mutable object as a key (its `hashCode` changes after insertion) makes entries “disappear” — you can’t `get` them because the bucket index changed. Always use **immutable keys** (`String`, `Long`, records). Overriding `equals` but not `hashCode` breaks map lookups — a classic prod bug.

6. Most Asked Interview Questions

- Explain HashMap internal working. *(hash → bucket → list/tree → equals)*
- What is load factor? Why 0.75? *(space/time tradeoff; higher = more collisions, lower = wasted space)*
- How/when does resize happen? *(size > cap*0.75 → double + rehash)*
- What is treeification and its thresholds? *(list→tree at 8 & cap≥64)*
- Why must capacity be a power of 2? *(`(n-1) & hash` works as fast modulo)*
- equals/hashCode contract? *(equal → same hash; unequal may collide)*
- What happens with a null key? *(bucket 0)*
- Is HashMap thread-safe? What breaks under concurrency? *(no; resize can corrupt/infinite-loop in Java 7; use CHM)*
- **Follow-up:** difference between Java 7 and 8 HashMap? *(Java 8 added treeification, tail-insert to avoid the Java 7 resize infinite-loop)*

7. Interview Traps

- Overriding `equals` without `hashCode` (or vice versa).
- Saying collisions become a tree immediately (needs length ≥8 **and** capacity ≥64).
- Saying HashMap maintains order (it doesn’t — use `LinkedHashMap`).
- Claiming HashMap put/get is always O(1) (worst case O(log n) after treeify, O(n) before Java 8).

8. Best Answer

> “`HashMap` is an array of buckets. I compute a spread hash (`h ^ h>>>16`) and index with `(n-1)&hash`. Collisions chain in a linked list; from Java 8, once a bucket has ≥8 nodes and the table is ≥64, it becomes a red-black tree for O(log n) worst case. It resizes — doubling capacity and rehashing — when size exceeds capacity × 0.75. Keys must be immutable with a consistent `equals`/`hashCode`, or lookups break.”

9. Coding Example — correct equals/hashCode

```java
public final class UserId {
    private final String tenant;
    private final long id;
    public UserId(String tenant, long id){ this.tenant = tenant; this.id = id; }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof UserId u)) return false;
        return id == u.id && tenant.equals(u.tenant);
    }
    @Override public int hashCode() { return Objects.hash(tenant, id); }
}
// Now safe as a HashMap key.
```

10. Follow-up Coding Questions

- Find the first non-repeating char using a `LinkedHashMap<Character,Integer>`.
- Group anagrams using `HashMap<String,List<String>>`.
- Implement a simple hash map from scratch (array + linked list + resize).

11. Summary

Array of buckets, spread hash, `(n-1)&hash` index, chaining → tree at 8/64, resize at 0.75. Immutable keys + correct equals/hashCode.

12. Cheat Sheet

```
default cap 16, LF 0.75, resize=double+rehash
index=(n-1)&hash ; hash=h^(h>>>16)
treeify: bucket>=8 AND cap>=64 ; untreeify<6
null key -> bucket 0 ; not thread-safe
equals true => hashCode equal (mandatory)
```

2.5 ConcurrentHashMap (CHM)

Core Concept & Internal Working

Thread-safe `Map` without locking the whole table. - **Java 7:** segment locking (default 16 segments) — lock striping. - **Java 8+:** dropped segments. Uses the **bucket array + CAS** for empty-bucket inserts and **`synchronized` on the bucket's head node** for collisions. Reads are lock-free (`volatile` nodes). Also treeifies like HashMap. - **No null keys or values** (ambiguity between "absent" and "null" in concurrent `get`). - `size()` is approximate under concurrency (uses `baseCount` + counter cells). - Atomic composites: `putIfAbsent`, `compute`, `computeIfAbsent`, `merge` — use these instead of check-then-act.

Memory Diagram

```
Java 8 CHM:
 bucket empty  -> CAS in new node (no lock)
 bucket has head-> synchronized(head){ append / update }   (fine-grained)
 reads          -> volatile, lock-free
```

Interview Q&A

- **HashMap vs Hashtable vs CHM?** HashMap not thread-safe; Hashtable synchronizes every method (whole-object lock, legacy, slow); CHM locks per-bucket → high concurrency.
- **Why no null in CHM?** Can't distinguish "key absent" from "mapped to null" without locking.
- **Why not just `Collections.synchronizedMap`?** That wraps a single lock — no concurrency; CHM allows concurrent reads and striped writes.
- **`computeIfAbsent` use?** Atomic lazy init of cache entries (avoids race of `if(!contains) put`).

Best Answer

> "CHM gives thread-safety with high throughput: in Java 8 it CASes into empty buckets and only `synchronized`-locks the individual bucket head on collision, while reads stay lock-free via volatile. It forbids nulls and offers atomic `computeIfAbsent`/`merge` so I don't do racy check-then-act. `Hashtable` locks the whole map — I never use it."

2.6 Sets — HashSet, LinkedHashSet, TreeSet

	HashSet	LinkedHashSet	TreeSet
Backing	HashMap	LinkedHashMap	TreeMap (red-black)
Order	none	insertion	sorted
Null	1 null	1 null	no null (NPE on compare)
get/add/contains	O(1)	O(1)	O(log n)

- **`HashSet` is literally a `HashMap`** with a dummy `PRESENT` value for every key. Same equals/hashCode rules apply.

- `TreeSet` needs elements `Comparable` or a `Comparator`; keeps sorted order; offers `first/last/ceiling/floor/headSet/tailSet`.
- `LinkedHashSet` = predictable iteration order (insertion), slightly more memory.

2.7 Queues & Deque — PriorityQueue, ArrayDeque

PriorityQueue

- A **binary min-heap** (array-backed). `peek` = min = O(1); `offer`/`poll` = O(log n).
- Ordering by natural order or a `Comparator`. **Not** sorted iteration — only the head is guaranteed smallest.
- Not thread-safe (use `PriorityBlockingQueue`).
- Classic uses: Dijkstra, top-K, task scheduling by priority.

ArrayDeque

- Resizable-array double-ended queue. **Preferred stack and queue** (faster than `Stack` and `LinkedList`; no synchronization overhead).
- `offerFirst/offerLast/pollFirst/pollLast`, or `push/pop` for stack semantics. No null allowed.

Memory Diagram — PriorityQueue heap

```
Array: [1, 3, 2, 7, 4]  represents min-heap:
              1
            /   \
           3     2
          / \
         7   4
parent(i)=(i-1)/2 ; children=2i+1, 2i+2 ; poll() removes root, sifts down
```

2.8 Maps — TreeMap, LinkedHashMap

TreeMap

- **Red-black tree**, sorted by key (natural or `Comparator`). O(log n) ops.
- `NavigableMap`: `firstKey/lastKey/ceilingKey/floorKey/headMap/tailMap/subMap`.
- No null keys. Use when you need **range queries** or sorted iteration.

LinkedHashMap

- HashMap + doubly-linked list across entries → predictable **insertion** (or **access**) order.
- **`accessOrder=true` + override `removeEldestEntry`** → a ready-made **LRU cache** (top interview coding ask).

```
class LRUCache<K,V> extends LinkedHashMap<K,V> {
    private final int cap;
    LRUCache(int cap){ super(cap, 0.75f, true); this.cap = cap; } // access-order
    @Override protected boolean removeEldestEntry(Map.Entry<K,V> e){ return size() > cap; }
}
```

2.9 Comparable vs Comparator

	Comparable	Comparator
Method	`compareTo(o)`	`compare(a,b)`
Where	inside the class	external/separate
Count	one “natural” order	many orders
Example	`String`, `Integer`	sort users by age then name

```
class Emp { String name; int age; }
List<Emp> list = ...;
list.sort(Comparator.comparingInt((Emp e) -> e.age)   // Comparator, Java 8 style
        .thenComparing(e -> e.name));
// reverse: .reversed()
```

- `compareTo/compare` return negative/zero/positive.
- **Trap:** `compareTo` inconsistent with `equals` breaks `TreeSet`/`TreeMap` (they use compare, not equals — two “equal by compare” elements are deduped!).

2.10 Iterator · Fail-Fast vs Fail-Safe · ConcurrentModificationException

Core Concept

- **Iterator** = cursor to traverse a collection; supports safe `remove()`.
- **Fail-fast** iterators (ArrayList, HashMap) track a **`modCount`**. If the collection is structurally modified during iteration (except via the iterator), the next `next()` throws **`ConcurrentModificationException`** (CME) — best-effort, not guaranteed.
- **Fail-safe** iterators (`CopyOnWriteArrayList`, `ConcurrentHashMap`) iterate over a **snapshot** or tolerate concurrent modification → no CME, but may not reflect latest writes.

Internal Working — modCount

```
expectedModCount = modCount (at iterator creation)
list.add(x) during loop -> modCount++
it.next() -> if (modCount != expectedModCount) throw ConcurrentModificationException
```

> CME can happen in a **single thread** too — e.g., adding/removing inside a for-each loop.

Best Answer & Fixes

> “Fail-fast iterators throw `ConcurrentModificationException` when they detect structural change via `modCount`, even single-threaded — like removing inside a for-each. I fix it with `Iterator.remove()`, `removeIf()`, collecting to a new list, or a concurrent/copy-on-write collection when I truly have multiple threads.”

```
// WRONG - throws CME:
for (String s : list) if (s.isBlank()) list.remove(s);
// RIGHT:
list.removeIf(String::isBlank);
```

2.11 Collections & Arrays Utility Classes

`Collections` (operates on Collection objects)

`sort`, `reverse`, `shuffle`, `binarySearch`, `max/min`, `frequency`, `unmodifiableList`, `synchronizedList/Map`, `emptyList`, `singletonList`.

`Arrays` (operates on arrays)

`sort`, `binarySearch`, `fill`, `copyOf`, `equals`, `asList`, `stream`, `toString`, `parallelSort`. - **Trap:** `Arrays.asList(arr)` returns a **fixed-size** list backed by the array — `add`/`remove` throw `UnsupportedOperationException`; changes write through to the array. For a mutable copy: `new ArrayList<>(Arrays.asList(arr))`. - `List.of(...)` / `Map.of(...)` (Java 9) → **immutable**; `add` throws.

2.12 Time & Space Complexity (must-memorize table)

Structure	get/contains	add	remove	Notes
ArrayList	O(1) index / O(n) search	amortized O(1) end, O(n) middle	O(n)	array copy on resize
LinkedList	O(n)	O(1) ends	O(1) w/ node	2 pointers/node
ArrayDeque	O(n) search	O(1) ends	O(1) ends	best stack/queue
HashMap/HashSet	O(1) avg, O(log n) treeified	O(1) avg	O(1) avg	0.75 LF
LinkedHashMap/Set	O(1)	O(1)	O(1)	keeps order
TreeMap/TreeSet	O(log n)	O(log n)	O(log n)	sorted, red-black
PriorityQueue	O(1) peek, O(n) contains	O(log n)	O(log n)	binary heap
ConcurrentHashMap	O(1) avg	O(1) avg	O(1) avg	lock-striped/CAS

Module 2 — Top 25 Interview Questions (senior answers)

1. **Is `Map` a `Collection`?** No — separate hierarchy.
2. **How does HashMap work internally?** hash → `(n-1)&hash` bucket → list/tree → equals.
3. **Load factor & default capacity?** 0.75, 16; resize doubles at size>cap*0.75.
4. **Treeification thresholds?** ≥8 in a bucket **and** cap≥64 → red-black tree; <6 untreeify.
5. **Why capacity power of 2?** `(n-1)&hash` acts as fast modulo and distributes evenly.
6. **equals/hashCode contract?** Equal objects must have equal hashCode; needed for map/set correctness.
7. **What if you override equals but not hashCode?** Lookups fail — objects land in wrong/duplicate buckets.
8. **HashMap vs Hashtable vs ConcurrentHashMap?** Unsynchronized vs whole-object-locked vs bucket-level/CAS.
9. **Why no null in ConcurrentHashMap?** Can't distinguish absent vs null concurrently.
10. **ArrayList vs LinkedList?** Array (O(1) get, cache-friendly) vs nodes (O(1) ends only).
11. **How does ArrayList grow?** 1.5x + arraycopy; amortized O(1) add.
12. **Vector vs ArrayList?** Vector synchronized + doubles; ArrayList not synced + 1.5x.
13. **HashSet vs TreeSet vs LinkedHashSet?** Unordered O(1) / sorted O(log n) / insertion-order.
14. **Comparable vs Comparator?** Natural single order in-class vs multiple external orders.
15. **compareTo inconsistent with equals — impact?** TreeSet/TreeMap drop "equal" elements.
16. **Fail-fast vs fail-safe?** modCount CME vs snapshot iteration.
17. **When does ConcurrentModificationException happen?** Structural change during iteration (even single-thread).
18. **How to remove during iteration?** `Iterator.remove()` / `removeIf`.

19. **PriorityQueue internals?** Binary min-heap; O(log n) offer/poll.
20. **Best stack/queue implementation?** `ArrayDeque` (not `Stack`/`LinkedList`).
21. **How to build an LRU cache?** `LinkedHashMap` access-order + `removeEldestEntry`.
22. **`Arrays.asList` gotcha?** Fixed-size, array-backed; wrap in `new ArrayList<>()`.
23. **Immutable collections?** `List.of`, `Map.of`, `Collections.unmodifiableList`.
24. **When TreeMap over HashMap?** Sorted keys / range queries (`ceilingKey`, `subMap`).
25. **Complexity of HashMap get worst case?** O(log n) after treeify (O(n) pre-Java 8).

Module 2 — Top Coding Questions

- Implement LRU cache (LinkedHashMap and manual HashMap+DLL).
- First non-repeating character (LinkedHashMap).
- Group anagrams; find duplicates; count word frequency.
- Top-K frequent elements (HashMap + PriorityQueue).
- Merge/sort by multiple fields with `Comparator.comparing().thenComparing()`.
- Detect and remove duplicates preserving order (LinkedHashSet).
- Implement your own HashMap (buckets + resize).

Module 2 — Common Follow-ups

- “Java 7 vs Java 8 HashMap?” (treeify + tail-insert fixed resize infinite loop.)
- “Why did old HashMap infinite-loop under concurrency?” (Java 7 head-insert reversed list on concurrent resize.)
- “How is CHM `size()` computed?” (baseCount + CounterCells; approximate.)
- “What happens to iteration order after resize?” (rehash changes bucket order; never rely on HashMap order.)

Module 2 — One-Page Cheat Sheet

```
Map != Collection. List(dup,ordered) Set(unique) Queue(FIFO) Deque(both)
ArrayList: dynamic array, 1.5x grow, O(1) get, O(n) middle
LinkedList: doubly-linked, O(1) ends; prefer ArrayList/ArrayDeque
HashMap: cap16 LF0.75, index=(n-1)&hash, hash=h^h>>>16, chain->tree(8 & cap>=64), resize=double+rehash
  immutable keys + equals/hashCode required; null key->bucket0
CHM: Java8 CAS empty + synchronized head; no nulls; computeIfAbsent atomic
Sets: HashSet=HashMap; LinkedHashSet=order; TreeSet=sorted O(log n)
Maps: TreeMap sorted(rbtree); LinkedHashMap=order/LRU(accessOrder+removeEldestEntry)
PriorityQueue=min-heap O(log n); ArrayDeque=best stack/queue
Comparable=natural(compareTo); Comparator=custom(compare, thenComparing, reversed)
fail-fast(modCount->CME) vs fail-safe(snapshot). remove via Iterator.remove/removeIf
Arrays.asList = fixed-size backed array; List.of = immutable
```

Module 2 — Mock Interview (answer, then continue)

1. “Walk me through `map.put(key, value)` end to end, including collision and resize.”
2. “You override `equals()` on an entity but forget `hashCode()`. What breaks and why?”
3. “How would you build an LRU cache in Java? Now make it thread-safe.”
4. “Explain how ConcurrentHashMap achieves thread-safety without locking the whole map.”
5. “Why is `ArrayDeque` preferred over `Stack` and `LinkedList`?”
6. “You get a `ConcurrentModificationException` while filtering a list in one thread. Why, and 3 ways to fix it?”

Model answers are in the sections above. Continue to Module 3 when ready.

Module 3 — Java 8+ (Lambdas, Functional Interfaces, Streams, Optional)

> Every modern Java interview has a "write this with streams" coding round. Service companies ask stream/lambda syntax; product companies ask stream *internals* and lazy evaluation.

Node.js bridge: Java streams ≈ JS array methods (`map`, `filter`, `reduce`) but **lazy** and one-shot. Lambdas ≈ arrow functions. Functional interfaces ≈ callback type signatures.

3.1 Lambdas & Functional Interfaces

1. Why Interviewers Ask This

Foundation for streams and modern Spring (functional endpoints, `Optional`, callbacks).

2. Core Concept

- A **functional interface** = exactly one abstract method (`@FunctionalInterface`). May have `default`/`static` methods.
- A **lambda** is a concise implementation of that single method: `(args) -> body`.
- **Method reference** = shorthand for a lambda that just calls an existing method: `String::toUpperCase`.

3. Internal Working

Lambdas are **not** anonymous inner classes. `javac` emits an `invokedynamic` bytecode; at first execution `LambdaMetafactory` spins up the implementation class (often reusing a singleton for stateless lambdas). Result: lower overhead and no extra `.class` file per lambda. A lambda captures **effectively final** local variables (copied), and `this` refers to the **enclosing** instance (unlike anonymous classes).

4. Memory Diagram

```
Runnable r = () -> log(x);   // x must be effectively final (captured by value)
   javac -> invokedynamic -> LambdaMetafactory -> synthetic impl (cached if stateless)
   'this' inside lambda == enclosing object (NOT a new anonymous instance)
```

5. The 4 core functional interfaces (memorize)

Interface	Signature	Meaning	Example
`Function<T,R>`	`R apply(T)`	transform	`s -> s.length()`
`Predicate<T>`	`boolean test(T)`	condition	`s -> s.isEmpty()`
`Consumer<T>`	`void accept(T)`	side effect	`System.out::println`
`Supplier<T>`	`T get()`	produce	`() -> new ArrayList<>()`

Also: `BiFunction`, `BinaryOperator`, `UnaryOperator`, `BiPredicate`, `BiConsumer`, and primitive variants (`IntFunction`, `ToIntFunction`, `IntPredicate`) to avoid boxing.

6. Most Asked Questions

- What is a functional interface? Name the built-in ones.
- Lambda vs anonymous class? *(this scope, invokedynamic, no separate class)*
- What is "effectively final"? *(captured locals can't be reassigned)*
- 4 kinds of method references? *(static `C::m`, instance-of-object `obj::m`, instance-of-type `C::m`, constructor `C::new`)*

7. Traps

- Thinking a lambda's `this` = the lambda (it's the enclosing object).
- Trying to mutate a captured local (compile error).
- Overusing lambdas where a named method is clearer.

8. Best Answer

> "A functional interface has one abstract method; a lambda implements it and compiles to `invokedynamic` via `LambdaMetafactory`, not an inner class — so `this` is the enclosing object and stateless lambdas are cached. Captured locals must be effectively final. I lean on `Function`, `Predicate`, `Consumer`, `Supplier` and method references for readability."

9. Coding Example

```
Function<String,Integer> len = String::length;              // method reference
Predicate<String> nonEmpty = s -> !s.isEmpty();
Supplier<List<String>> newList = ArrayList::new;            // constructor ref
Consumer<String> print = System.out::println;
BiFunction<Integer,Integer,Integer> add = Integer::sum;
System.out.println(len.andThen(n -> n * 2).apply("hello")); // 10
```

10. Follow-ups

- Compose predicates with `.and()/.or()/.negate()`.
- Write a generic `retry(Supplier<T>, int)` helper.

11 & 12. Summary + Cheat

FI = 1 abstract method; lambda = its impl via invokedynamic. Function/Predicate/Consumer/Supplier.

3.2 Streams — the flagship Java 8 topic

1. Why Interviewers Ask This

"Rewrite this loop with streams" and "what does this stream print" appear in nearly every coding round.

2. Core Concept

A **Stream** is a pipeline over a data source: **source → intermediate ops → terminal op**. Streams don't store data, don't mutate the source, and are **single-use** (consumed once).

3. Internal Working — laziness & fusion

- **Intermediate** ops (`map`, `filter`, `sorted`, `distinct`, `limit`, `flatMap`, `peek`) are **lazy** — they build a pipeline and do nothing until a terminal op runs.
- **Terminal** ops (`collect`, `forEach`, `reduce`, `count`, `findFirst`, `anyMatch`, `toList`) trigger execution.
- Elements are pushed **one at a time** through the whole chain (**loop fusion**) — not materialized between stages. This enables **short-circuiting** (`findFirst`, `limit`, `anyMatch` stop early) and lazy `map` calls that never run for skipped elements.
- **`parallelStream()`** splits work across the common ForkJoinPool. Use only for large, CPU-bound, stateless, associative work — and never mutate shared state.

4. Memory Diagram

```
source -> filter -> map -> collect
[1,2,3,4] each element flows fully through the chain before the next:
  1 -> filter(even?) drop
  2 -> filter ok -> map(*10)=20 -> collector
  3 -> drop
  4 -> ok -> 40 -> collector      => [20,40]
Nothing runs until collect() (terminal). limit(1) would stop after first match.
```

5. Key operations

- `map` — 1:1 transform. `flatMap` — 1:many, flattens nested streams (`List<List<T>>` → `Stream<T>`).
- `filter` — keep matching. `reduce` — fold to a single value.
- `collect(Collectors...)` — `toList`, `toSet`, `toMap`, `joining`, `groupingBy`, `partitioningBy`, `counting`, `summingInt`, `averagingDouble`.
- `groupingBy` — `Map<K, List<V>>` (SQL GROUP BY). `partitioningBy` — `Map<Boolean, List<V>>` (split by predicate).

6. Most Asked Questions

- Intermediate vs terminal ops? Give examples.
- Are streams lazy? What triggers execution?
- `map` vs `flatMap`?
- Can you reuse a stream? *(No — `IllegalStateException`.)*
- When parallel streams? Risks? *(shared mutable state, small data overhead, ordering.)*
- `Collectors.toMap` duplicate key behavior? *(throws unless you pass a merge function.)*

7. Traps

- Reusing a consumed stream.
- Side effects in `map`/`peek` (should be pure).
- `toMap` without a merge function on duplicate keys → `IllegalStateException`.
- Assuming parallel streams are always faster (usually slower for small/IO work).
- `forEach` on a parallel stream expecting order (use `forEachOrdered`).

8. Best Answer

> “A stream is a lazy pipeline: intermediate ops just describe work and only run when a terminal op pulls elements through, one at a time with loop fusion, which lets short-circuit ops like `findFirst` stop early. `map` is 1:1, `flatMap` flattens nested structures. I use `groupingBy`/`partitioningBy` for aggregation and avoid parallel streams unless the workload is large, CPU-bound, and stateless.”

9. Coding Example

```
record Emp(String name, String dept, int salary) {}
List<Emp> emps = ...;

// group salaries by department
Map<String, List<Emp>> byDept = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept));

// average salary per department
Map<String, Double> avg = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept, Collectors.averagingInt(Emp::salary)));

// highest paid per department
Map<String, Optional<Emp>> top = emps.stream()
    .collect(Collectors.groupingBy(Emp::dept,
             Collectors.maxBy(Comparator.comparingInt(Emp::salary))));

// names of employees earning > 100k, sorted, comma-joined
String csv = emps.stream()
    .filter(e -> e.salary() > 100_000)
    .map(Emp::name).sorted()
    .collect(Collectors.joining(", "));

// partition into high/low earners
Map<Boolean, List<Emp>> parts = emps.stream()
    .collect(Collectors.partitioningBy(e -> e.salary() > 100_000));

// flatMap: all distinct chars across names
List<Character> chars = emps.stream()
    .flatMap(e -> e.name().chars().mapToObj(c -> (char) c))
    .distinct().toList();
```

10. Follow-up Coding Questions

- Frequency count with `Collectors.groupingBy(x, counting())`.
- Second highest salary using streams.
- Sum of squares of even numbers with `reduce`/`sum`.
- Flatten `List<List<Integer>>` to a sorted distinct list.
- Convert `List<Emp>` to `Map<name, salary>` handling duplicate names.

11 & 12. Summary + Cheat

Lazy pipeline, single-use, terminal triggers. map/flatMap/filter/reduce/collect; groupingBy/partitioningBy for aggregation.

3.3 Optional

Core Concept & Internal Working

A container that may or may not hold a non-null value — designed to make “absence” explicit at the type level and reduce NPEs. Intended primarily as a **return type**, not for fields or parameters.

API you must know

`Optional.of(x)` (x must be non-null), `ofNullable(x)`, `empty()`, `isPresent()/isEmpty()`, `get()` (avoid), `orElse(default)`, `orElseGet(supplier)`, `orElseThrow()`, `map`, `flatMap`, `filter`, `ifPresent(consumer)`, `ifPresentOrElse(...)`.

Best practices / traps

- **Never** call `get()` without checking — defeats the purpose and NPE-equivalent.
- **`orElse` vs `orElseGet`:** `orElse(expensive())` **always** evaluates the argument even when present; `orElseGet(() -> expensive())` is lazy. Use `orElseGet` for costly defaults.

- Don't use `Optional` for fields, method params, or collections (use empty collections instead).
- Don't do `if (opt.isPresent()) opt.get()` — use `map`/`ifPresent`/`orElse`.

```
// Spring Data returns Optional:
User user = repo.findById(id)
    .orElseThrow(() -> new NotFoundException("user " + id));

String city = repo.findById(id)
    .map(User::getAddress)
    .map(Address::getCity)
    .orElse("UNKNOWN");                // null-safe chain, no NPE
```

Best Answer

> "`Optional` makes absence explicit in the return type so callers must handle it. I chain `map`/`flatMap` for null-safe navigation and use `orElseThrow`/`orElseGet` — never bare `get()`. I avoid it for fields and parameters; that's an anti-pattern."

3.4 Method References (quick reference)

Kind	Syntax	Lambda equivalent
Static	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Instance of a *particular* object	`System.out::println`	`x -> System.out.println(x)`
Instance of an *arbitrary* object of a type	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

Module 3 — Top 25 Interview Questions (senior answers)

1. **Functional interface?** One abstract method; `@FunctionalInterface`.
2. **Built-in functional interfaces?** Function, Predicate, Consumer, Supplier (+ Bi/primitive variants).
3. **Lambda vs anonymous class?** invokedynamic, enclosing `this`, no extra class.
4. **Effectively final?** Captured locals can't be reassigned.
5. **Method reference kinds?** static, bound instance, unbound instance, constructor.
6. **What is a Stream?** Lazy single-use pipeline over a source.
7. **Intermediate vs terminal ops?** Lazy builders vs execution triggers.
8. **Is stream lazy? Proof?** Yes — `peek`/`map` don't run until terminal; short-circuits stop early.
9. **map vs flatMap?** 1:1 vs 1:many flatten.
10. **reduce use?** Fold to single value with identity + accumulator (+ combiner for parallel).
11. **collect / Collectors?** toList/toMap/joining/groupingBy/partitioningBy/counting.
12. **groupingBy vs partitioningBy?** Arbitrary key map vs boolean 2-way split.
13. **Can you reuse a stream?** No — IllegalStateException.
14. **Parallel stream risks?** Shared mutable state, overhead, ordering, common pool starvation.
15. **toMap duplicate keys?** Throws unless merge function provided.
16. **Stream vs Collection?** Computation pipeline vs data storage.
17. **Optional purpose?** Explicit absence, fewer NPEs; return type only.
18. **orElse vs orElseGet?** Eager arg vs lazy supplier.
19. **Why not Optional fields/params?** Overhead + not designed for it; use empty collections.
20. **findFirst vs findAny?** Deterministic first vs any (parallel-friendly).

21. **peek use/abuse?** Debugging only; not for side effects driving logic.
22. **mapToInt/IntStream?** Avoid boxing; get `sum/average/summaryStatistics`.
23. **Collectors.joining?** Concatenate with delimiter/prefix/suffix.
24. **How does parallelStream split?** Spliterator + common ForkJoinPool.
25. **Infinite streams?** `Stream.iterate/generate` + `limit` (must short-circuit).

Module 3 — Top Coding Questions

- Group employees by dept; average/max salary per dept.
- Find duplicates / first non-repeating char with streams.
- Second highest number; top-K frequent words.
- Flatten nested lists (`flatMap`); merge maps.
- Word frequency count; sort a `Map` by value.
- Partition numbers into even/odd; sum with `reduce`.

Module 3 — Common Follow-ups

- “Show it with a `for` loop, then streams — which is clearer here?”
- “Would you parallelize this? Why/why not?”
- “How does `Collectors.toMap` behave on collisions and how do you fix it?”

Module 3 — One-Page Cheat Sheet

```
Functional interface = 1 abstract method. Lambda -> invokedynamic (not inner class), enclosing this
Function(apply) Predicate(test) Consumer(accept) Supplier(get)
Method refs: Class::static, obj::method, Class::instanceMethod, Class::new
Stream = lazy, single-use pipeline. Intermediate(map/filter/flatMap/sorted/limit) lazy;
Terminal(collect/reduce/forEach/count/findFirst/anyMatch) triggers.
map=1:1, flatMap=flatten. Collectors: toList/toMap(merge!)/joining/groupingBy/partitioningBy/counting
Optional: return type only; map/flatMap/orElseThrow; orElse(eager) vs orElseGet(lazy); never bare get()
parallelStream only for big CPU-bound stateless work
```

Module 3 — Mock Interview (answer, then continue)

1. “Given `List<Order>`, produce total revenue per customer, sorted descending, as a `LinkedHashMap`.”
2. “Explain, step by step, what elements flow through `stream().filter(...).map(...).findFirst()` and why `map` may run fewer times than the list size.”
3. “`orElse` vs `orElseGet` — when does it matter in production?”
4. “Rewrite a nested loop that flattens and dedups a list of lists using streams.”
5. “When would you refuse to use a parallel stream?”

Continue to Module 4 when ready.

Module 4 — Exception Handling

> A guaranteed topic. “Checked vs unchecked”, “can `finally` override a return”, and “how do you design exceptions in a REST API” show up constantly.

Node.js bridge: JS has one untyped `Error` and `try/catch`. Java has a **typed hierarchy**, a compiler-enforced **checked** category, and `try-with-resources` for deterministic cleanup.

4.1 Exception Hierarchy, Checked vs Unchecked, Error

1. Why Interviewers Ask This

Tests whether you understand recoverability and API design, not just `try/catch` syntax.

2. Core Concept

```
                  Throwable
             +--------+--------+
           Error               Exception
        (don't catch)     +-------+---------+
    OutOfMemoryError   (checked)        RuntimeException (unchecked)
    StackOverflowError  IOException       NullPointerException
                        SQLException      IllegalArgumentException
                        ClassNotFound     IllegalStateException
                                          ArithmeticException
                                          IndexOutOfBounds
```

- **Checked** (compile-time): extend `Exception` (not `RuntimeException`). Compiler forces `throws` or `catch`. For **recoverable** external conditions (IO, DB).
- **Unchecked** (runtime): extend `RuntimeException`. For **programming errors** (bad args, null, illegal state). Not enforced.
- **Error**: serious JVM problems (`OutOfMemoryError`, `StackOverflowError`). Don’t catch/recover.

3. Internal Working

`throw` creates the exception object (capturing the **stack trace** at construction — expensive) and unwinds the stack frame by frame looking for a matching `catch`. If none is found in the thread, the thread’s `UncaughtExceptionHandler` runs and the thread dies. Building the stack trace (`fillInStackTrace`) is the costly part — never use exceptions for control flow.

4. Memory Diagram

```
throw new X()  --> unwind stack:
  frame f3  (no catch) pop
  frame f2  (catch X?) yes -> handle
  frame f1
If unwinds past main() -> UncaughtExceptionHandler -> thread terminates
```

5. Real Production Example

In Spring, a `@RestControllerAdvice` with `@ExceptionHandler` maps exceptions to HTTP: `EntityNotFoundException → 404`, `MethodArgumentNotValidException → 400`, everything else → 500. Checked `IOException` from a file/HTTP call gets wrapped in a domain `RuntimeException` so service signatures stay clean.

6. Most Asked Questions

- Checked vs unchecked — definition and when to use each?
- Is `NullPointerException` checked or unchecked? *(unchecked)*

- Can you catch an `Error`? Should you? *(can, shouldn't)*
- Difference between `throw` and `throws`?
- What's the cost of throwing? *(stack trace capture.)*

7. Traps

- Catching `Exception` or `Throwable` broadly and swallowing it (empty catch).
- Using exceptions for normal control flow.
- Catching then rethrowing while losing the cause (always chain: `new X(msg, cause)`).
- Confusing `throw` (statement) with `throws` (declaration).

8. Best Answer

> "Checked exceptions extend `Exception` and are compiler-enforced for recoverable external failures like IO; unchecked extend `RuntimeException` for programming bugs. `Error` is for JVM-level failures I never catch. Throwing captures a stack trace, so it's expensive — I never use it for control flow, always chain the cause, and in Spring I centralize mapping in a `@RestControllerAdvice`."

9. Coding Example

```
public class OrderService {
    Order place(String id) {
        try {
            return gateway.charge(id);          // may throw IOException (checked)
        } catch (IOException e) {
            // wrap checked -> domain unchecked, preserve cause
            throw new PaymentFailedException("charge failed for " + id, e);
        }
    }
}
class PaymentFailedException extends RuntimeException {
    PaymentFailedException(String m, Throwable cause){ super(m, cause); }
}
```

10. Follow-ups

- When would you make a custom exception checked vs unchecked? *(unchecked by default in modern code/Spring)*
- How does exception chaining help debugging? *(preserves root cause + full trace)*

11 & 12. Summary + Cheat

Checked=recoverable/enforced; Unchecked=bugs; Error=JVM. Chain causes; don't swallow.

4.2 try-catch-finally · try-with-resources · throw/throws

finally semantics (favorite trap)

- `finally` **always** runs — even on `return`/`throw` in try/catch. Only skipped by `System.exit()` or JVM crash.
- A `return` (or `throw`) in `finally` **overrides** any prior return/exception — an anti-pattern that swallows exceptions. Never `return` from `finally`.

```
int f() {
    try { return 1; }
    finally { return 2; }   // returns 2, swallows the 1 -> DON'T do this
}
```

try-with-resources (the modern way)

Any `AutoCloseable`/`Closeable` declared in the `try(...)` header is closed automatically in reverse order, even on exception — replacing verbose `finally { conn.close(); }`.

```
try (var conn = dataSource.getConnection();
     var ps   = conn.prepareStatement(SQL)) {
    ps.execute();
}   // ps then conn closed automatically, exceptions suppressed & attached
```

- Exceptions thrown by `close()` are **suppressed** and attached to the primary (`getSuppressed()`).
- Cleaner and leak-proof vs manual `finally`.

throw vs throws · multi-catch

- `throw` — actually throws an instance: `throw new IllegalArgumentException("x")`.
- `throws` — declares a method *may* throw: `void read() throws IOException`.
- **Multi-catch**: `catch (IOException | SQLException e)` — one block, `e` is effectively final; can't be a supertype/subtype pair.

4.3 Custom Exceptions

Best practice

```
public class InsufficientFundsException extends RuntimeException {   // unchecked by default
    private final String accountId;
    public InsufficientFundsException(String accountId, Throwable cause) {
        super("Insufficient funds for account " + accountId, cause);  // message + cause
        this.accountId = accountId;
    }
    public String getAccountId() { return accountId; }
}
```

- Extend `RuntimeException` unless callers can meaningfully recover (then `Exception`).
- Include context (ids, values) and preserve the cause.
- Give it a clear name ending in `Exception`.
- In Spring, map with `@ResponseStatus` or `@ExceptionHandler`.

Spring global handler (production pattern)

```
@RestControllerAdvice
class ApiExceptionHandler {
    @ExceptionHandler(EntityNotFoundException.class)
    ResponseEntity<ApiError> notFound(EntityNotFoundException e){
        return ResponseEntity.status(404).body(new ApiError("NOT_FOUND", e.getMessage()));
    }
    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ApiError> badRequest(MethodArgumentNotValidException e){
        return ResponseEntity.badRequest().body(new ApiError("VALIDATION", e.getMessage()));
    }
}
```

Module 4 — Top 25 Interview Questions (senior answers)

1. **Exception hierarchy?** Throwable → Error / Exception → RuntimeException.
2. **Checked vs unchecked?** Compiler-enforced recoverable vs programming bugs.
3. **Is Error catchable?** Technically yes; never recover from it.
4. **throw vs throws?** Throw an instance vs declare possibility.
5. **Does finally always run?** Yes, except `System.exit`/JVM crash.
6. **Can finally override return?** Yes — anti-pattern; avoid returning in finally.

7. **try-with-resources?** Auto-closes AutoCloseable in reverse order; suppresses close exceptions.
8. **Suppressed exceptions?** From `close()`, attached via `getSuppressed()`.
9. **Multi-catch rules?** `A | B`, effectively final, no sub/supertype overlap.
10. **NPE checked/unchecked?** Unchecked (RuntimeException).
11. **Custom exception: checked or unchecked?** Prefer unchecked in modern/Spring code.
12. **Exception chaining?** Pass cause to constructor to keep root cause.
13. **Cost of throwing?** Stack trace capture; don't use for control flow.
14. **Catch order?** Subclass before superclass or compile error.
15. **finally vs finalize vs final?** Cleanup block vs deprecated GC hook vs modifier.
16. **Rethrow best practice?** Wrap with cause, add context, don't swallow.
17. **Checked exception in a stream lambda?** Not allowed — wrap in unchecked or use helper.
18. **What is a stack trace?** Call chain captured at construction.
19. **When wrap checked → unchecked?** To keep service signatures clean / cross layers.
20. **Handling exceptions in Spring REST?** `@RestControllerAdvice` + `@ExceptionHandler`.
21. **`@ResponseStatus`?** Maps an exception to an HTTP status.
22. **Global vs local handling?** Centralize cross-cutting; local for specific recovery.
23. **Difference: `printStackTrace` vs logging?** Use a logger; never `printStackTrace` in prod.
24. **Can constructors throw?** Yes.
25. **What happens to uncaught exception in a thread?** UncaughtExceptionHandler; thread dies.

Module 4 — Top Coding Questions

- Implement a domain exception hierarchy + Spring `@RestControllerAdvice`.
- Convert legacy `finally { close(); }` to try-with-resources.
- Write a method that wraps a checked exception in a lambda (`Function` wrapper).
- Predict output of tricky try/finally/return snippets.

Module 4 — Common Follow-ups

- "Would you make this exception checked? Why not?"
- "What happens to a `close()` exception in try-with-resources?"
- "How do exceptions behave inside a stream pipeline?"

Module 4 — One-Page Cheat Sheet

```
Throwable -> Error (don't catch) | Exception -> checked (IOException) / RuntimeException (unchecked)
Checked = recoverable, compiler-enforced (throws/catch). Unchecked = bugs.
throw=instance ; throws=declaration ; multi-catch (A|B)
finally always runs (except System.exit); never return in finally
try-with-resources: AutoCloseable, reverse close, suppressed exceptions
Custom: extend RuntimeException, add context, chain cause
Spring: @RestControllerAdvice + @ExceptionHandler / @ResponseStatus
Never: swallow, use for control flow, printStackTrace in prod
```

Module 4 — Mock Interview (answer, then continue)

1. "Design the exception strategy for a REST API — which layers throw, which translate to HTTP?"
2. "What does a `try { return 1; } finally { return 2; }` return, and why is it dangerous?"

3. “A checked `IOException` bubbles up from a stream lambda — how do you handle it?”
4. “Walk me through what try-with-resources compiles to and what suppressed exceptions are.”
5. “When do you create a checked vs unchecked custom exception?”

Continue to Module 5 when ready.

Module 5 — Multithreading & Concurrency

> High priority for backend roles. Product companies dig into `volatile`, the memory model, and deadlock; service companies ask lifecycle, `synchronized`, and `ExecutorService`.

Node.js bridge: Node is single-threaded with an event loop; you offload with worker threads or async IO. Java uses **real OS threads** with shared memory — which means you personally own **visibility** and **atomicity** correctness. This is the biggest mental shift from Node.

5.1 Process vs Thread · Thread Lifecycle

Process vs Thread

- **Process** — own memory/address space; heavy; isolated.
- **Thread** — lightweight unit inside a process; **shares heap/metaspace**, has its own **stack** + PC. Cheap to create, but shared mutable state needs synchronization.

Thread Lifecycle (states in `Thread.State`)

```
NEW --start()--> RUNNABLE --scheduler--> (running)
  RUNNABLE <-> BLOCKED (waiting for monitor lock)
  RUNNABLE <-> WAITING (wait()/join()/park(), no timeout)
  RUNNABLE <-> TIMED_WAITING (sleep(t)/wait(t))
  --> TERMINATED (run() returns/throws)
```

- `RUNNABLE` covers both “ready” and “running” (JVM doesn’t distinguish).
- `BLOCKED` = waiting to acquire a monitor; `WAITING` = waiting for another thread’s signal.

5.2 Runnable · Callable · ExecutorService · ThreadPool

Runnable vs Callable

	Runnable	Callable
Method	`void run()`	`V call() throws Exception`
Returns	nothing	a value
Checked exceptions	no	yes
With executor	`execute`/`submit`	`submit` → `Future<V>`

ExecutorService & thread pools (never `new Thread()` in prod)

- Creating threads manually is unbounded → OOM under load. Use a **pool** to cap and reuse threads.
- `Executors` factories: `newFixedThreadPool(n)`, `newCachedThreadPool()`, `newSingleThreadExecutor()`, `newScheduledThreadPool(n)`, `newVirtualThreadPerTaskExecutor()` (Java 21).
- **Prefer constructing `ThreadPoolExecutor` directly** in production for a **bounded queue** and explicit rejection policy — `newFixedThreadPool` uses an *unbounded* `LinkedBlockingQueue` that can OOM.

Internal Working — ThreadPoolExecutor

```
submit(task):
  if activeThreads < corePoolSize      -> create core thread, run
  else if queue not full               -> enqueue task
  else if activeThreads < maxPoolSize  -> create non-core thread
  else                                 -> RejectedExecutionHandler (Abort/CallerRuns/Discard)
idle non-core threads die after keepAliveTime
```

Memory Diagram

```
[submit] -> core threads busy? -> [ bounded queue ] full? -> spawn up to max -> full? -> REJECT
                                                                              |
                                                                   RejectedExecutionHandler
```

Coding Example

```
ExecutorService pool = new ThreadPoolExecutor(
    4, 8, 60, TimeUnit.SECONDS,
    new ArrayBlockingQueue<>(100),                 // bounded queue (backpressure)
    new ThreadPoolExecutor.CallerRunsPolicy());    // graceful rejection

Future<Integer> f = pool.submit(() -> expensiveCompute());  // Callable
Integer result = f.get(2, TimeUnit.SECONDS);                // blocks with timeout
pool.shutdown();                                            // then awaitTermination
```

Best Answer

> "I never spawn raw threads; I use an `ExecutorService`. In production I build a `ThreadPoolExecutor` with core/max sizes, a **bounded** queue for backpressure, and an explicit rejection policy — because `Executors.newFixedThreadPool` hides an unbounded queue that can OOM. `Runnable` returns nothing; `Callable` returns a `Future`."

5.3 synchronized · volatile · Atomic — visibility & atomicity

The Java Memory Model (JMM) — the concept behind everything

Each thread may cache variables in registers/CPU caches. Without synchronization, one thread's write may **never become visible** to another. The JMM defines **happens-before**: `synchronized` release → acquire and `volatile` write → read establish visibility ordering.

synchronized

- Mutual exclusion + visibility. Acquires an object's **monitor** (intrinsic lock). Only one thread holds it; others `BLOCKED`.
- On method (`synchronized void m()`) locks `this` (or the Class for static); on block `synchronized(lock){}` locks a chosen object.
- Reentrant (same thread can re-acquire). Guarantees both **atomicity** of the block and **visibility** of all changes made inside.

volatile

- Guarantees **visibility** and ordering (no caching, no reordering across it) — but **NOT atomicity** of compound ops (`count++` is read-modify-write → still racy even if `volatile`).
- Use for **flags** (`volatile boolean running`) and the double-checked-locking singleton reference.

Atomic classes

- `AtomicInteger/AtomicLong/AtomicReference` provide **lock-free** atomic ops via **CAS** (Compare-And-Swap, a CPU instruction). `incrementAndGet()` is atomic without locking.

- Higher throughput than `synchronized` under contention for simple counters. `LongAdder` scales even better for hot counters.

Memory Diagram

```
count++  (NOT atomic):
  T1 read 5 | T2 read 5 | T1 write 6 | T2 write 6   -> lost update!
Fixes: synchronized{count++}  OR  AtomicInteger.incrementAndGet() (CAS retry loop)
volatile boolean flag: write by T1 immediately visible to T2 (no cache staleness)
```

Comparison

Tool	Atomicity	Visibility	Cost
`volatile`	No (single read/write only)	Yes	cheap
`synchronized`	Yes	Yes	lock (blocking)
`Atomic*` (CAS)	Yes (single var)	Yes	lock-free, fast

5.4 Locks (java.util.concurrent.locks)

- **`ReentrantLock`** — explicit lock/unlock (always in `finally`), supports `tryLock(timeout)`, fairness, interruptible waits — more flexible than `synchronized`.
- **`ReadWriteLock` / `StampedLock`** — many readers OR one writer; great for read-heavy caches.
- Always: `lock.lock(); try { ... } finally { lock.unlock(); }`.

```
private final ReentrantLock lock = new ReentrantLock();
void safe() {
    lock.lock();
    try { /* critical section */ }
    finally { lock.unlock(); }   // MUST unlock in finally
}
```

5.5 Deadlock & Race Condition

Race Condition

Outcome depends on thread timing (e.g., lost update on `count++`, check-then-act on a map). Fix with synchronization, atomics, or immutable/confined state.

Deadlock — the 4 Coffman conditions

Mutual exclusion + hold-and-wait + no preemption + **circular wait**. Break any one.

```
T1: lock A ... wait for B
T2: lock B ... wait for A     -> both stuck forever
```

Prevention: acquire locks in a **global consistent order**; use `tryLock` with timeout; minimize lock scope; prefer higher-level concurrency utilities.

Best Answer

> “A race condition is when correctness depends on timing — like two threads doing `count++` and losing an update; I fix it with atomics or synchronization. A deadlock needs all four Coffman conditions; the practical fix is a consistent global lock ordering or `tryLock` with a timeout so a thread backs off instead of waiting forever.”

5.6 CompletableFuture (basic)

Async composition without blocking — Java's answer to JS Promises.

```
CompletableFuture
    .supplyAsync(() -> fetchUser(id), pool)          // async, on a pool
    .thenApply(User::profile)                        // transform (map)
    .thenCompose(p -> fetchOrdersAsync(p))           // chain another future (flatMap)
    .thenCombine(fetchRecommendations(id), Result::merge)  // join two futures
    .exceptionally(ex -> Result.empty())             // error handling
    .thenAccept(this::respond);                      // consume
```

- `thenApply` = map; `thenCompose` = flatMap; `thenCombine` = zip; `exceptionally`/`handle` = error handling.
- Non-blocking — don't call `.get()` on the hot path; compose instead.
- Always supply your own executor for blocking work (default is the common ForkJoinPool).

Node bridge: `supplyAsync` ≈ `new Promise`, `thenApply` ≈ `.then`, `exceptionally` ≈ `.catch`.

Module 5 — Top 25 Interview Questions (senior answers)

1. **Process vs thread?** Own memory vs shared heap, own stack.
2. **Thread states?** NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
3. **Runnable vs Callable?** void vs returns value/throws checked → Future.
4. **Why ExecutorService over new Thread?** Reuse, bounded, backpressure, lifecycle.
5. **newFixedThreadPool risk?** Unbounded queue → OOM; build ThreadPoolExecutor with bounded queue.
6. **ThreadPoolExecutor params?** core, max, keepAlive, queue, rejection handler.
7. **Rejection policies?** Abort, CallerRuns, Discard, DiscardOldest.
8. **synchronized — what does it guarantee?** Atomicity + visibility via monitor.
9. **volatile — what it does/doesn't?** Visibility+ordering, NOT compound atomicity.
10. **Is `count++` atomic with volatile?** No — read-modify-write race.
11. **Atomic classes / CAS?** Lock-free atomic single-var ops.
12. **synchronized vs Lock?** Implicit vs explicit (tryLock, fairness, interruptible).
13. **What is reentrancy?** Same thread can re-acquire a held lock.
14. **wait/notify vs sleep?** Releases lock + needs monitor vs just pauses, keeps lock.
15. **Race condition?** Timing-dependent incorrect result.
16. **Deadlock & 4 conditions?** Mutual exclusion, hold-wait, no preemption, circular wait.
17. **Prevent deadlock?** Global lock ordering, tryLock timeout, minimal scope.
18. **Happens-before?** JMM ordering from locks/volatile that guarantees visibility.
19. **ThreadLocal use & risk?** Per-thread state; leak in pools if not `remove()`.
20. **CompletableFuture vs Future?** Composable/non-blocking vs blocking `get`.
21. **thenApply vs thenCompose?** map vs flatMap.
22. **ConcurrentHashMap vs synchronizedMap?** Bucket-level/CAS vs single lock.
23. **Producer-consumer?** `BlockingQueue` (put/take block).
24. **CountDownLatch vs CyclicBarrier vs Semaphore?** One-shot wait / reusable barrier / permits.
25. **Virtual threads (Java 21)?** Cheap JVM-scheduled threads for high-concurrency IO (`newVirtualThreadPerTaskExecutor`).

Module 5 — Top Coding Questions

- Thread-safe counter (synchronized vs AtomicInteger vs LongAdder).

- Producer-consumer with `BlockingQueue`.
- Thread-safe singleton (double-checked locking with volatile; or enum/holder idiom).
- Print odd/even alternately with two threads (wait/notify or Lock+Condition).
- Run N tasks in parallel and combine results with CompletableFuture.
- Reproduce and then fix a deadlock.

Module 5 — Common Follow-ups

- “Why is `volatile` not enough for a counter?”
- “Your fixed pool works in dev but OOMs in prod under a spike — why?”
- “How would you cap concurrency to 10 outbound calls?” (Semaphore / bounded pool.)

Module 5 — One-Page Cheat Sheet

```
Process=own memory; Thread=shared heap+own stack
States: NEW/RUNNABLE/BLOCKED/WAITING/TIMED_WAITING/TERMINATED
Runnable(void) vs Callable(V, Future). Use ExecutorService, bounded queue, rejection policy
synchronized = atomicity+visibility (monitor, reentrant)
volatile = visibility only (NOT count++). Atomic/CAS = lock-free atomic single var
Lock: lock/unlock in finally; tryLock timeout; ReadWriteLock for read-heavy
Race = timing bug. Deadlock = 4 Coffman -> fix: global lock order / tryLock
CompletableFuture: supplyAsync/thenApply(map)/thenCompose(flatMap)/thenCombine/exceptionally
Singleton: enum or volatile double-checked locking
ThreadLocal: remove() in pools to avoid leaks. Java21 virtual threads for IO concurrency
```

Module 5 — Mock Interview (answer, then continue)

1. “Two threads increment a shared `int` a million times each; the total is wrong. Explain why and give three fixes with trade-offs.”
2. “Design a thread pool config for a service that calls a slow downstream API — sizes, queue, rejection?”
3. “Explain `volatile` vs `synchronized` vs `AtomicInteger` and when you’d pick each.”
4. “Show me a deadlock and fix it two different ways.”
5. “Rewrite three blocking downstream calls to run concurrently and merge with CompletableFuture.”

Continue to Module 6 when ready.

Module 6 — Spring Framework (IoC, DI, Beans)

> Highest priority. This is the core of every Spring interview. If you can explain IoC, DI, and the bean lifecycle crisply, you clear most Spring rounds.

Node.js bridge: In Express you `require()` and `new` your dependencies yourself. Spring *inverts* that: a **container** creates and wires your objects for you (like a built-in, type-safe DI framework — think NestJS, which was inspired by Spring/Angular).

6.1 Spring Architecture & IoC (Inversion of Control)

1. Why Interviewers Ask This

IoC/DI is the whole point of Spring. “What is IoC?” and “What is the difference between IoC and DI?” are near-guaranteed openers.

2. Core Concept

- **IoC (Inversion of Control)** — you don’t create/wire dependencies; the **container** does. Control of object lifecycle is inverted from your code to the framework.
- **DI (Dependency Injection)** — the *technique* IoC uses: dependencies are **injected** (constructor/setter/field) rather than looked up or `new`ed.
- **IoC Container** — `ApplicationContext` (`BeanFactory` is the lower-level base) creates, configures, wires, and manages **beans**.

> IoC is the principle; DI is the implementation of that principle. This exact distinction is a frequent follow-up.

3. Internal Working — how the container boots

```
1. Read metadata: @ComponentScan finds @Component/@Service/... ; @Configuration @Bean methods
2. Build BeanDefinitions (class, scope, dependencies, init/destroy) in a registry
3. Instantiate singletons (eagerly by default)
4. Populate dependencies (resolve @Autowired) -> dependency graph
5. Run BeanPostProcessors (this is where AOP proxies, @Transactional, @Async wrap beans)
6. Call init callbacks (@PostConstruct / InitializingBean / init-method)
7. Container ready; beans served on demand
```

BeanDefinitions are metadata; the container uses them to build the object graph, detecting cycles and ordering creation.

4. Memory Diagram

```
           ApplicationContext (IoC container)
   +--------------------------------------------------+
   | BeanDefinition registry:                          |
   |   OrderService -> needs PaymentGateway            |
   |   StripeGateway -> (no deps)                      |
   +--------------------------------------------------+
                    | instantiate + wire
   [StripeGateway singleton] <--injected-- [OrderService singleton]
```

5. Real Production Example

Your `OrderService` needs a `PaymentGateway`. You never write `new StripeGateway()`. You declare the dependency; Spring injects the right implementation, and in tests injects a mock — that’s the productivity and testability win of IoC.

6. Most Asked Questions

- What is IoC? Difference between IoC and DI? *(principle vs technique)*
- What is the IoC container / `ApplicationContext` vs `BeanFactory`? *(ApplicationContext = superset: events, i18n, AOP, eager singletons)*
- How does Spring know what to inject? *(by type, then by name/qualifier)*
- Benefits of DI? *(loose coupling, testability, single responsibility)*

7. Traps

- Saying IoC and DI are the same thing.
- Not knowing `ApplicationContext` extends `BeanFactory`.
- Thinking Spring uses reflection "magic" with no metadata (it builds BeanDefinitions).

8. Best Answer

> "IoC means the framework, not my code, controls object creation and wiring. DI is how it does that — injecting collaborators instead of me `new`-ing them. The `ApplicationContext` reads bean metadata, builds a dependency graph, instantiates singletons, injects dependencies, wraps them with proxies via BeanPostProcessors for cross-cutting concerns, and runs init callbacks. The payoff is loose coupling and trivially mockable tests."

9. Coding Example

```
@Service
class OrderService {
    private final PaymentGateway gateway;          // depends on abstraction
    OrderService(PaymentGateway gateway) {         // constructor injection (preferred)
        this.gateway = gateway;
    }
}
@Component
class StripeGateway implements PaymentGateway { /* ... */ }
// Spring wires StripeGateway into OrderService automatically.
```

10. Follow-ups

- What if two beans implement `PaymentGateway`? *(`@Primary` or `@Qualifier`)*
- How do you inject a mock in a unit test? *(constructor injection → pass the mock directly, no Spring needed)*

11 & 12. Summary + Cheat

IoC = container controls; DI = injection technique. ApplicationContext ⊃ BeanFactory.

6.2 Dependency Injection — Constructor vs Field vs Setter

Types

- **Constructor injection (preferred)** — dependencies final, immutable, mandatory; enables testing without Spring; fails fast if missing; detects circular deps at startup.
- **Setter injection** — optional/changeable dependencies.
- **Field injection (`@Autowired` on a field)** — concise but **discouraged**: can't make fields final, hides dependencies, hard to unit test without reflection, allows circular deps to slip through.

Best Answer

> "I use constructor injection: it makes dependencies explicit and final, guarantees a fully initialized object, works in plain unit tests without the Spring context, and surfaces circular dependencies at startup. Field injection is convenient but untestable and hides the contract, so I avoid it."

```
// PREFERRED - constructor injection (no @Autowired needed if single constructor)
@Service
class UserService {
    private final UserRepository repo;
    private final EmailClient email;
    UserService(UserRepository repo, EmailClient email) { this.repo = repo; this.email = email; }
}
```

Circular dependency

`A → B → A` via constructor injection **fails at startup** (`BeanCurrentlyInCreationException`). Fixes: redesign (best), `@Lazy` on one, or setter injection. Interviewers love this.

6.3 Beans, Scopes, Lifecycle

What is a Bean?

Any object instantiated, assembled, and managed by the Spring IoC container. Defined via stereotype annotations (`@Component` and friends) or `@Bean` methods in `@Configuration`.

Bean Scopes

Scope	Meaning
singleton (default)	one shared instance per container
prototype	new instance every injection/`getBean`
request	one per HTTP request (web)
session	one per HTTP session (web)
application	one per ServletContext

Trap: singleton beans are **shared** — keep them **stateless**. A prototype injected into a singleton is created **once** (at wiring) unless you use `ObjectProvider`/`@Lookup`/scoped proxy.

Bean Lifecycle (guaranteed question)

```
Instantiate -> Populate properties (DI) -> *Aware callbacks (BeanNameAware...) ->
BeanPostProcessor.postProcessBeforeInitialization ->
@PostConstruct -> InitializingBean.afterPropertiesSet() -> custom init-method ->
BeanPostProcessor.postProcessAfterInitialization (AOP proxy created here) ->
[BEAN READY / IN USE] ->
@PreDestroy -> DisposableBean.destroy() -> custom destroy-method
```

- `@PostConstruct` — run init logic after DI (cache warmup, validation).
- `@PreDestroy` — cleanup (only reliably called for singletons, not prototypes).
- **BeanPostProcessor** is where Spring wraps beans in **AOP proxies** — this is how `@Transactional`, `@Async`, `@Cacheable`, and security work.

Memory Diagram

```
new Bean() -> inject deps -> @PostConstruct -> [proxy? wrap via BPP] -> READY
                                                                      |
                                     (@Transactional method call goes through proxy)
```

6.4 Stereotype & Wiring Annotations

Annotation	Meaning
`@Component`	generic Spring-managed bean
`@Service`	business logic (semantic `@Component`)
`@Repository`	data access; also **translates persistence exceptions** to `DataAccessException`
`@Controller`	web MVC controller (returns views)
`@RestController`	`@Controller` + `@ResponseBody` (returns JSON/body)
`@Configuration`	class of `@Bean` definitions
`@Bean`	method producing a container-managed bean (for 3rd-party classes you can't annotate)
`@ComponentScan`	discover stereotypes in packages
`@Autowired`	inject by type
`@Qualifier("name")`	disambiguate among multiple candidates
`@Primary`	default candidate when multiple exist

`@Component` vs `@Bean`

- `@Component` — class-level, auto-detected by scanning; for **your** classes.
- `@Bean` — method-level in `@Configuration`; for **third-party** classes or when you need construction logic.

`@Qualifier` vs `@Primary`

Multiple `PaymentGateway` beans → ambiguity. `@Primary` picks a default globally; `@Qualifier` picks a specific one at the injection point (overrides `@Primary`).

```
@Configuration
class AppConfig {
    @Bean @Primary PaymentGateway stripe() { return new StripeGateway(); }
    @Bean @Qualifier("paypal") PaymentGateway paypal() { return new PaypalGateway(); }
}
@Service
class Checkout {
    Checkout(@Qualifier("paypal") PaymentGateway gw) { /* gets paypal */ }
}
```

`@Configuration` internal detail (nice senior point)

`@Configuration` classes are themselves CGLIB-proxied so that inter-`@Bean` method calls return the **same singleton** rather than a new instance (unlike plain `@Bean` in a `@Component`, "Lite" mode). A frequent deep follow-up.

Module 6 — Top 25 Interview Questions (senior answers)

1. **IoC vs DI?** Principle (container controls) vs technique (inject dependencies).
2. **ApplicationContext vs BeanFactory?** Superset: eager singletons, events, i18n, AOP, auto-BPP.
3. **What is a bean?** Container-managed object built from a BeanDefinition.
4. **Bean scopes?** singleton (default), prototype, request, session, application.
5. **Default scope pitfall?** Singleton shared → must be stateless.
6. **Bean lifecycle?** Instantiate → DI → Aware → BPP-before → @PostConstruct → afterPropertiesSet → init → BPP-after → ready → @PreDestroy → destroy.
7. **@PostConstruct/@PreDestroy?** Init after DI / cleanup before destroy.
8. **What is a BeanPostProcessor?** Hook that wraps beans (AOP proxies) around init.

9. **Constructor vs field vs setter injection?** Immutable/testable vs concise-but-discouraged vs optional.
10. **Why avoid field injection?** No final, hidden deps, hard to test, hides cycles.
11. **Circular dependency handling?** Constructor cycle fails fast; fix by redesign/@Lazy/setter.
12. **@Component vs @Service vs @Repository vs @Controller?** Semantics; @Repository adds exception translation.
13. **@Controller vs @RestController?** Views vs JSON body.
14. **@Component vs @Bean?** Auto-scanned class vs method for third-party/factory.
15. **@Autowired resolution order?** By type → by qualifier/name; @Primary as default.
16. **@Qualifier vs @Primary?** Point-specific vs global default; qualifier wins.
17. **@ComponentScan?** Discovers stereotypes in base packages.
18. **Is @Autowired required?** Optional on a single constructor since Spring 4.3.
19. **Prototype in singleton problem?** Injected once; use ObjectProvider/@Lookup/scoped proxy.
20. **How is @Transactional applied?** Via AOP proxy created by a BeanPostProcessor.
21. **@Configuration CGLIB proxy?** Ensures @Bean calls return the same singleton.
22. **Lazy vs eager beans?** Singletons eager by default; `@Lazy` defers creation.
23. **How to define a bean for a 3rd-party class?** `@Bean` in `@Configuration`.
24. **Spring vs Spring Boot?** Framework (DI/MVC) vs opinionated auto-config on top.
25. **How to unit test a service?** Constructor-inject mocks; no context needed.

Module 6 — Top Coding Questions

- Wire two implementations and select with `@Qualifier`/`@Primary`.
- Demonstrate the bean lifecycle with `@PostConstruct`/`@PreDestroy` logging.
- Reproduce a circular dependency and fix it.
- Register a third-party client (e.g., `RestTemplate`/`WebClient`) as a `@Bean`.

Module 6 — Common Follow-ups

- “What actually happens at `context.getBean(OrderService.class)`?”
- “Why must singletons be stateless?”
- “How does `@Transactional` work under the hood?” (proxy + BeanPostProcessor.)

Module 6 — One-Page Cheat Sheet

```
IoC = container controls creation/wiring. DI = injection technique. ApplicationContext ⊃ BeanFactory
Lifecycle: instantiate->DI->Aware->BPP.before->@PostConstruct->afterPropertiesSet->init->BPP.after-
>READY->@PreDestroy->destroy
BeanPostProcessor wraps AOP proxies (@Transactional/@Async/@Cacheable)
Scopes: singleton(default, stateless!) prototype request session application
Injection: constructor(preferred, final, testable) > setter(optional) > field(avoid)
Circular ctor dep -> fails fast -> redesign/@Lazy/setter
@Component(class,scan) vs @Bean(method,3rd-party). @Repository=exception translation
@Autowired by type; @Qualifier(point) beats @Primary(default)
@Configuration is CGLIB-proxied -> @Bean calls share singleton
```

Module 6 — Mock Interview (answer, then continue)

1. “Explain IoC vs DI to a Node developer, with a concrete example.”
2. “Walk through the full bean lifecycle and tell me exactly where `@Transactional` gets applied.”

3. “You have two `PaymentGateway` beans; injection fails. Two ways to fix it and which you’d choose.”
4. “Why is constructor injection better than field injection? Convince me.”
5. “A prototype-scoped bean injected into a singleton isn’t behaving as expected — why, and how do you fix it?”

Continue to Module 7 when ready.

Module 7 — Spring Boot

> Highest priority. The "how does auto-configuration work" and "trace a request through the DispatcherServlet" questions are asked at every level. Know the request flow cold.

Node.js bridge: Spring Boot ≈ an opinionated, batteries-included Express/Nest setup: embedded server (like `app.listen`), auto-wired middleware, config profiles, health endpoints — minus the manual plumbing.

7.1 Spring Boot Architecture & Auto-Configuration

1. Why Interviewers Ask This

Auto-configuration is *the* Spring Boot feature. "What is `@SpringBootApplication`?" and "how does auto-config decide what to configure?" are guaranteed.

2. Core Concept

Spring Boot = Spring + **auto-configuration** + **starters** + **embedded server** + **opinionated defaults**. It removes boilerplate XML/config so you "just run".

`@SpringBootApplication` = `@Configuration` + `@ComponentScan` + **`@EnableAutoConfiguration`**.

3. Internal Working — auto-configuration mechanism

```
1. @EnableAutoConfiguration triggers AutoConfigurationImportSelector
2. It reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
   (older: spring.factories) from every starter jar -> list of @AutoConfiguration classes
3. Each auto-config class is GUARDED by @Conditional annotations:
      @ConditionalOnClass       (is H2/Tomcat/Jackson on the classpath?)
      @ConditionalOnMissingBean (did the user NOT define their own?)
      @ConditionalOnProperty    (is a property set?)
4. Conditions that pass -> beans registered (DataSource, DispatcherServlet, ObjectMapper...)
5. Your beans always win (@ConditionalOnMissingBean backs off if you defined one)
```

So "convention over configuration" = *classpath presence + conditions* decide what gets wired.

4. Memory Diagram

```
classpath has spring-boot-starter-web
   -> Tomcat + Spring MVC jars present
   -> WebMvcAutoConfiguration @ConditionalOnClass(DispatcherServlet) PASSES
   -> registers DispatcherServlet, Jackson ObjectMapper, embedded Tomcat
   (unless you defined your own -> @ConditionalOnMissingBean backs off)
```

5. Starter Dependencies

Curated, version-aligned dependency bundles: `spring-boot-starter-web` (MVC + Tomcat + Jackson), `-data-jpa` (Hibernate + JPA), `-security`, `-actuator`, `-test`. The **parent BOM** manages compatible versions so you don't resolve conflicts manually.

6. Most Asked Questions

- What is `@SpringBootApplication`? *(3 annotations)*
- How does auto-configuration work? *(imports + @Conditional)*
- What are starters? Why? *(curated, versioned dependency sets)*
- How do you override an auto-configured bean? *(define your own → `@ConditionalOnMissingBean` backs off)*
- How to exclude an auto-config? *(`@SpringBootApplication(exclude=DataSourceAutoConfiguration.class)`)*
- How to debug what got auto-configured? *(`--debug` → Conditions Evaluation Report)*

7. Traps

- Saying auto-config is "magic" — it's conditional bean registration from imports files.
- Thinking starters contain code (they're mostly dependency aggregators).
- Not knowing your own bean overrides the auto-configured one.

8. Best Answer

> "`@SpringBootApplication` bundles `@Configuration`, `@ComponentScan`, and `@EnableAutoConfiguration`. At startup, `AutoConfigurationImportSelector` loads auto-config classes listed in each starter's imports file. Each is guarded by `@Conditional` checks — `@ConditionalOnClass`, `@ConditionalOnMissingBean`, `@ConditionalOnProperty` — so beans are only created when the right libraries are present and I haven't defined my own. That's why adding `starter-web` gives me an embedded Tomcat and JSON mapping with zero config, but my own `ObjectMapper` bean always wins."

9. Coding Example

```
@SpringBootApplication
public class App {
    public static void main(String[] args) { SpringApplication.run(App.class, args); }

    @Bean                                   // overrides Boot's auto-configured ObjectMapper
    ObjectMapper objectMapper() {
        return new ObjectMapper().findAndRegisterModules()
            .disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
    }
}
```

10. Follow-ups

- Write a custom auto-configuration with `@ConditionalOnProperty`.
- How does the embedded Tomcat start? *(ServletWebServerFactory bean)*

11 & 12. Summary + Cheat

Auto-config = imports files + @Conditional. Starters = versioned deps. Your bean wins.

7.2 DispatcherServlet & the Request Lifecycle (know this cold)

Core Concept

The **`DispatcherServlet`** is the **front controller** — a single servlet that receives every HTTP request and orchestrates handling.

Internal Working — full request flow

```
HTTP request
  -> Servlet container (embedded Tomcat) -> Filter chain (Security, encoding, CORS)
  -> DispatcherServlet
       1. HandlerMapping        : find controller method for URL+verb (@RequestMapping)
       2. HandlerAdapter        : invoke it
       3. HandlerInterceptor    : preHandle()
       4. Argument resolvers    : bind @RequestBody/@PathVariable/@RequestParam (+ validation)
       5. Controller method runs -> returns object / ResponseEntity
       6. HttpMessageConverter  : serialize return value to JSON (Jackson)  [@ResponseBody path]
          (or ViewResolver -> render a view, for @Controller)
       7. HandlerInterceptor    : postHandle / afterCompletion
       8. @ExceptionHandler / HandlerExceptionResolver if anything threw
  -> Filter chain (response) -> Tomcat -> HTTP response
```

Memory Diagram

```
[Client] -> [Tomcat] -> [Filters] -> [DispatcherServlet]
                                         |-> HandlerMapping -> Controller
                                         |-> MessageConverter (JSON)
                                         |-> ExceptionResolver
                     <- response <--------+
```

Best Answer

> "Every request hits the embedded Tomcat, passes the servlet filter chain (security, CORS), then the `DispatcherServlet` front controller. It uses a `HandlerMapping` to find the `@RequestMapping` method, a `HandlerAdapter` to invoke it, argument resolvers to bind and validate the body/params, runs my controller, and an `HttpMessageConverter` (Jackson) to serialize the return value to JSON. Exceptions are routed to `@ExceptionHandler`. Filters are outside the DispatcherServlet; interceptors are inside it."

Filter vs Interceptor (common follow-up): Filters are servlet-level (before/after DispatcherServlet, work on raw request/response, e.g., security, logging). Interceptors are Spring MVC-level (have access to the handler/model, run around controller execution).

7.3 Configuration — properties, YAML, Profiles, @ConfigurationProperties

- **`application.properties` / `application.yml`** — externalized config. YAML is hierarchical/nicer for nested config; both are equivalent.
- **Profiles** — environment-specific config: `application-dev.yml`, `application-prod.yml`; activate with `spring.profiles.active=prod` (env var / CLI). `@Profile("prod")` conditionally registers beans.
- **Property precedence** (high→low): command-line args → OS env vars → profile-specific files → `application.yml` → defaults. (Know that env/CLI override files.)
- **`@Value("${db.url}")`** — inject a single property.
- **`@ConfigurationProperties(prefix="app")`** — type-safe binding of a group of properties to a POJO (preferred for structured config; supports validation and relaxed binding).

```
@ConfigurationProperties(prefix = "payment")
@Validated
public record PaymentProps(@NotBlank String apiKey, @Min(1) int timeoutMs) {}
// payment.api-key / payment.timeout-ms bind automatically (relaxed binding)
```

@Value vs @ConfigurationProperties: single value + SpEL vs grouped, type-safe, validated, relaxed-binding. Prefer `@ConfigurationProperties` for anything structured.

7.4 REST APIs, Validation, Exception Handling, Logging, Actuator

REST controllers

```
@RestController
@RequestMapping("/api/v1/users")
class UserController {
    private final UserService service;
    UserController(UserService service){ this.service = service; }

    @GetMapping("/{id}")
    User get(@PathVariable Long id){ return service.get(id); }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    User create(@Valid @RequestBody CreateUserRequest req){ return service.create(req); }

    @GetMapping
    Page<User> list(@RequestParam(defaultValue="0") int page,
                    @RequestParam(defaultValue="20") int size){
        return service.list(PageRequest.of(page, size));
    }
}
```

Validation

`@Valid` + Bean Validation (`jakarta.validation`) annotations (`@NotNull`, `@NotBlank`, `@Email`, `@Min`, `@Size`). Invalid `@RequestBody` → `MethodArgumentNotValidException` → map to **400**.

Exception handling (production pattern)

`@RestControllerAdvice` + `@ExceptionHandler` for centralized error responses (see Module 4).

Logging

SLF4J API + Logback (default). Configure levels per package in properties. Use structured/JSON logs + correlation ids (MDC) in microservices.

Actuator (operational endpoints)

`/actuator/health` (liveness/readiness for k8s), `/metrics`, `/info`, `/env`, `/loggers` (change log level at runtime), `/prometheus` (with Micrometer). Expose selectively and secure them — never expose `/env`/`/heapdump` publicly.

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,prometheus
  endpoint:
    health:
      probes:
        enabled: true   # /health/liveness & /health/readiness
```

Module 7 — Top 25 Interview Questions (senior answers)

1. **What is Spring Boot vs Spring?** Opinionated auto-config + starters + embedded server on top of Spring.
2. **@SpringBootApplication =?** @Configuration + @ComponentScan + @EnableAutoConfiguration.
3. **How does auto-configuration work?** Imports files + @Conditional guards.
4. **@ConditionalOnClass/OnMissingBean/OnProperty?** Presence/user-override/property gating.
5. **How to override an auto-configured bean?** Define your own → OnMissingBean backs off.
6. **Exclude auto-config?** `exclude = DataSourceAutoConfiguration.class`.

7. **What are starters?** Curated versioned dependency bundles.
8. **Embedded server?** Tomcat by default (Jetty/Undertow optional); no WAR needed.
9. **What is the DispatcherServlet?** Front controller orchestrating request handling.
10. **Full request lifecycle?** Tomcat → filters → DispatcherServlet → HandlerMapping → adapter → controller → MessageConverter → response.
11. **Filter vs Interceptor?** Servlet-level vs Spring MVC-level (handler-aware).
12. **How is JSON produced?** HttpMessageConverter (Jackson) on @ResponseBody.
13. **@Controller vs @RestController?** View vs body (JSON).
14. **application.properties vs yml?** Flat vs hierarchical; equivalent.
15. **Profiles?** Env-specific config/beans via `spring.profiles.active` + `@Profile`.
16. **Property precedence?** CLI > env > profile file > application.yml > defaults.
17. **@Value vs @ConfigurationProperties?** Single/SpEL vs grouped/type-safe/validated.
18. **How do you validate requests?** `@Valid` + Bean Validation → 400 on failure.
19. **Centralized exception handling?** `@RestControllerAdvice` + `@ExceptionHandler`.
20. **What is Actuator?** Production endpoints: health, metrics, info, loggers.
21. **k8s probes?** `/health/liveness` & `/health/readiness`.
22. **How to see what auto-configured?** Run with `--debug` → conditions report.
23. **How does Boot start Tomcat?** `ServletWebServerFactory` bean during context refresh.
24. **CommandLineRunner/ApplicationRunner?** Run code after startup.
25. **How to build a runnable artifact?** Fat/uber JAR via `spring-boot-maven-plugin`.

Module 7 — Top Coding Questions

- Build a paginated, validated CRUD REST controller + global exception handler.
- Bind config with `@ConfigurationProperties` + validation.
- Add a custom health indicator and a Micrometer metric.
- Write a `HandlerInterceptor` (or Filter) that logs latency + correlation id.

Module 7 — Common Follow-ups

- “Where exactly is the request body deserialized and validated?”
- “How would you run different config in dev vs prod?”
- “How do you expose metrics to Prometheus safely?”

Module 7 — One-Page Cheat Sheet

```
Boot = Spring + auto-config + starters + embedded Tomcat + defaults
@SpringBootApplication = @Configuration + @ComponentScan + @EnableAutoConfiguration
Auto-config: AutoConfiguration.imports + @ConditionalOnClass/OnMissingBean/OnProperty; your bean wins
Request: Tomcat -> Filters -> DispatcherServlet -> HandlerMapping -> HandlerAdapter ->
         controller -> HttpMessageConverter(Jackson) -> response ; @ExceptionHandler on error
Filter=servlet level ; Interceptor=MVC handler level
Config: yml/properties, profiles (spring.profiles.active), precedence CLI>env>profile>file
@Value(single) vs @ConfigurationProperties(grouped/typed/validated)
Validation: @Valid -> MethodArgumentNotValidException -> 400
Actuator: /health(liveness,readiness) /metrics /info /loggers /prometheus (secure them!)
```

Module 7 — Mock Interview (answer, then continue)

1. “Trace a `POST /api/v1/users` with a JSON body from socket to response, naming every component.”
2. “Explain auto-configuration deeply — how does Boot decide to create a `DataSource`?”
3. “How do you override the default Jackson `ObjectMapper`, and why does yours win?”
4. “Dev uses H2, prod uses Postgres — how do you configure that cleanly?”
5. “Which Actuator endpoints do you expose in prod and how do you secure them?”

Continue to Module 8 when ready.

Module 8 — Spring Data JPA & Hibernate

> Highest priority. The **N+1 problem**, **lazy vs eager**, **transactions**, and **entity lifecycle** are asked in nearly every backend Spring interview. This is where seniors are separated from juniors.

Node.js bridge: JPA/Hibernate is like a much more powerful Sequelize/TypeORM: it maps objects↔tables, tracks changes automatically (dirty checking), and has a first-level cache (persistence context) that Node ORMs mostly lack.

8.1 Hibernate Architecture & the Persistence Context

1. Why Interviewers Ask This

Understanding the persistence context explains dirty checking, first-level cache, lazy loading, and the N+1 problem — all in one model.

2. Core Concept

- **JPA** = the specification (annotations, `EntityManager`). **Hibernate** = the most common implementation. **Spring Data JPA** = a layer that generates repository implementations over JPA.
- **`EntityManager`** manages a **persistence context** — a first-level cache of managed entities within a transaction.
- **`SessionFactory`**/**`EntityManagerFactory`** — expensive, thread-safe, one per app. **`Session`**/**`EntityManager`** — cheap, per-transaction, not thread-safe.

3. Internal Working

```
Transaction begins -> EntityManager creates a Persistence Context (1st-level cache)
  find(id)  -> checks context first (cache hit = no SQL); else SELECT, store in context
  changes to managed entities are TRACKED (snapshots)
Transaction commit -> FLUSH: Hibernate compares snapshots -> generates UPDATE/INSERT/DELETE
                      (dirty checking) -> clears context
```

4. Memory Diagram

```
+----------------- Transaction / Persistence Context -----------------+
| managed:  User#1 (snapshot: name="A")  <- same object returned every |
|                                            find(1) in this tx        |
| you call user.setName("B")  -> no SQL yet                            |
| commit -> flush -> dirty check finds name changed -> UPDATE users... |
+----------------------------------------------------------------------+
```

5. Real Production Example

You load a `User`, set a field, and never call `save()` — it still updates on commit because of **dirty checking**. Two `findById(1)` calls in one transaction hit the DB **once** (first-level cache returns the same instance).

8.2 Entity Lifecycle (states)

```
   new User()      persist()/save()      commit/close        find()
 TRANSIENT --------> MANAGED --------> DETACHED           (managed again)
 (no id, not         (tracked in       (context closed,    merge() -> back to MANAGED
  in context)         context)          changes not tracked)
                       |
                   remove() -> REMOVED -> DELETE on flush
```

- **Transient** — new object, not associated with a context, no DB row.

- **Managed/Persistent** — tracked; changes auto-flushed (dirty checking).
- **Detached** — was managed, context closed; changes NOT tracked until `merge()`.
- **Removed** — scheduled for deletion.

Trap: `LazyInitializationException` happens when you access a lazy association on a **detached** entity (outside the transaction/session) — very common in DTO mapping after the transaction closes.

8.3 First-Level Cache · Dirty Checking

- **First-level cache** = the persistence context. Always on, per-transaction. Repeated `find(id)` in one tx = one SQL. (Second-level cache is optional, across transactions — e.g., Ehcache.)
- **Dirty checking** = at flush, Hibernate compares each managed entity to its loaded snapshot and auto-generates `UPDATE` for changed fields. No explicit `save()` needed for managed entities.

8.4 Lazy vs Eager Loading + The N+1 Problem (the #1 JPA interview topic)

Lazy vs Eager

- **LAZY** — association loaded on first access (proxy until then). Default for `@OneToMany`/`@ManyToMany`.
- **EAGER** — loaded immediately with the parent. Default for `@ManyToOne`/`@OneToOne`.
- **Best practice:** make (almost) everything **LAZY**; fetch what you need explicitly with fetch joins/entity graphs. Eager collections cause hidden joins and over-fetching.

The N+1 Problem

```
Query 1:  SELECT * FROM orders;                        -- returns N orders
Then for EACH order, lazy access to order.getCustomer():
  SELECT * FROM customers WHERE id=?    (x N)           -- N extra queries
=> 1 + N queries  (N+1)  -> catastrophic latency at scale
```

Fixes

1. **Fetch join** — `@Query("select o from Order o join fetch o.customer")` → one SQL.
2. **`@EntityGraph`** — declaratively fetch associations on a repository method.
3. **Batch fetching** — `@BatchSize(size=50)` / `hibernate.default_batch_fetch_size` → `IN (...)`.
4. **DTO projection** — select only needed columns via constructor expression/projection.

```
@EntityGraph(attributePaths = "customer")
@Query("select o from Order o where o.status = :s")
List<Order> findByStatus(@Param("s") Status s);   // single join, no N+1
```

Best Answer

> “The N+1 problem is one query to load N parents, then N more to lazily load each parent’s association — 1+N round-trips that kill latency. I default associations to LAZY to avoid accidental over-fetching, then fetch exactly what I need with a `join fetch`, an `@EntityGraph`, batch fetching, or a DTO projection. I diagnose it by enabling `show-sql`/statistics and watching for repeated identical selects.”

8.5 Repositories — CrudRepository, JpaRepository, Paging & Sorting

```
Repository (marker)
  CrudRepository<T,ID>        : save, findById, findAll, delete, count
    PagingAndSortingRepository: findAll(Pageable), findAll(Sort)
      JpaRepository<T,ID>      : + flush, saveAndFlush, batch, JPA-specific
```

- **Derived queries**: `findByEmailAndStatus(...)` — Spring parses the method name into a query.
- **`@Query`** — JPQL or native (`nativeQuery=true`) for complex queries.
- **Paging & Sorting**: `Page<T> findAll(Pageable)`, `PageRequest.of(page, size, Sort.by("name"))`. `Page` runs an extra count query; `Slice` doesn't (cheaper when you don't need total count).

```
interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    Page<User> findByStatus(Status status, Pageable pageable);
    @Query("select u from User u where u.age > :age")
    List<User> olderThan(@Param("age") int age);
}
```

8.6 Transactions (@Transactional) · Dirty Checking

Core Concept & Internal Working

`@Transactional` is applied via an **AOP proxy** (Module 6): the proxy opens a transaction before the method, commits on normal return, and **rolls back on unchecked exceptions**. - **Default rollback:** only `RuntimeException`/`Error`. Checked exceptions do **NOT** roll back unless you set `rollbackFor = Exception.class`. (Huge trap.) - **Propagation:** `REQUIRED` (default, join or create), `REQUIRES_NEW` (suspend + new), `NESTED`, `SUPPORTS`, `MANDATORY`. - **Isolation:** `READ_COMMITTED` (typical default), `REPEATABLE_READ`, `SERIALIZABLE` — controls dirty/non-repeatable/phantom reads. - **`readOnly = true`** — optimization hint; skips dirty checking/flush for read queries.

Self-invocation trap (asked often)

Calling a `@Transactional` method from another method **in the same class** bypasses the proxy → **no transaction**. The call must come through the injected bean/proxy. Fix: move it to another bean or use self-injection.

```
@Service
class TransferService {
    @Transactional(rollbackFor = Exception.class)
    public void transfer(long from, long to, long cents) {
        accounts.debit(from, cents);
        accounts.credit(to, cents);   // if this throws, the debit rolls back
    }
}
```

Best Answer

> "`@Transactional` works through a Spring AOP proxy that begins a transaction, commits on success, and rolls back on unchecked exceptions by default — so I add `rollbackFor` for checked ones. I mark read methods `readOnly` to skip dirty checking. Two gotchas: self- invocation bypasses the proxy so the annotation is ignored, and `REQUIRES_NEW` suspends the outer transaction for an independent commit."

8.7 Relationships & Cascade Types

Mappings

- **`@OneToOne`** — user↔profile. Watch eager default; consider `@MapsId` for shared PK.
- **`@OneToMany` / `@ManyToOne`** — customer↔orders. Own the FK on the **`@ManyToOne`** (child) side; use `mappedBy` on the parent. Make collections LAZY.
- **`@ManyToMany`** — students↔courses via a join table. Prefer modeling the join table as its own entity when it has attributes.

Cascade Types

`PERSIST`, `MERGE`, `REMOVE`, `REFRESH`, `DETACH`, `ALL`. Cascade propagates operations from parent to children (e.g., saving an `Order` saves its `OrderLines`). **`orphanRemoval=true`** deletes children removed from the collection. **Trap:** `CascadeType.ALL` + `REMOVE` on a shared association can delete more than you intend.

```
@Entity
class Order {
    @OneToMany(mappedBy = "order", cascade = CascadeType.ALL, orphanRemoval = true,
               fetch = FetchType.LAZY)
    private List<OrderLine> lines = new ArrayList<>();

    @ManyToOne(fetch = FetchType.LAZY)     // override eager default -> avoid N+1
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

Module 8 — Top 25 Interview Questions (senior answers)

1. **JPA vs Hibernate vs Spring Data JPA?** Spec vs implementation vs repository abstraction.
2. **What is the persistence context?** Per-tx first-level cache of managed entities.
3. **Entity states?** Transient, Managed, Detached, Removed.
4. **What is dirty checking?** Auto UPDATE on flush by comparing snapshots.
5. **First-level vs second-level cache?** Per-tx (always on) vs cross-tx (optional).
6. **Lazy vs eager + defaults?** Lazy (collections) vs eager (@ManyToOne/@OneToOne); prefer lazy.
7. **LazyInitializationException cause/fix?** Access lazy field after tx closed; fetch join/@EntityGraph/DTO.
8. **N+1 problem + fixes?** 1+N queries; fetch join, @EntityGraph, batch size, DTO.
9. **Fetch join vs EntityGraph?** JPQL join fetch vs declarative attributePaths.
10. **CrudRepository vs JpaRepository?** Basic CRUD vs +batch/flush/JPA features.
11. **Derived query methods?** Method name parsed into query.
12. **@Query JPQL vs native?** Entity query language vs raw SQL (`nativeQuery=true`).
13. **Paging & sorting?** `Pageable`/`Sort`; `Page` (count) vs `Slice` (no count).
14. **@Transactional internals?** AOP proxy begin/commit/rollback.
15. **Default rollback rule?** Only unchecked; add `rollbackFor` for checked.
16. **Propagation types?** REQUIRED, REQUIRES_NEW, NESTED, SUPPORTS, MANDATORY.
17. **Isolation levels?** READ_COMMITTED/REPEATABLE_READ/SERIALIZABLE (reads).
18. **readOnly benefit?** Skips dirty checking/flush.
19. **Self-invocation problem?** Same-class call bypasses proxy → no tx.
20. **Cascade types?** PERSIST/MERGE/REMOVE/REFRESH/DETACH/ALL.
21. **orphanRemoval vs REMOVE?** Removes children detached from collection vs cascades delete.
22. **Who owns the FK in @OneToMany?** The @ManyToOne (child) side; parent uses mappedBy.
23. **save vs saveAndFlush?** Defer to commit vs flush immediately.

24. **Optimistic vs pessimistic locking?** `@Version` (no lock, retry) vs DB row lock.
25. **How to detect N+1 in prod?** `show-sql`, Hibernate statistics, APM traces.

Module 8 — Top Coding Questions

- Model `Customer` 1— *`Order`* —* `Product` with correct fetch/cascade.
- Fix an N+1 with a fetch join / `@EntityGraph`.
- Write a paginated + sorted repository method returning `Page<DTO>`.
- Implement optimistic locking with `@Version` and handle the conflict.
- Write a `@Transactional` money transfer with correct rollback semantics.

Module 8 — Common Follow-ups

- "Why did your UPDATE fire without calling save()?" (dirty checking.)
- "Why does this throw LazyInitializationException in the controller?" (detached entity.)
- "Why didn't the transaction roll back on a checked exception?" (default rule.)

Module 8 — One-Page Cheat Sheet

```
JPA(spec) / Hibernate(impl) / Spring Data JPA(repos)
Persistence context = per-tx 1st-level cache; dirty checking auto-UPDATE on flush
States: Transient -> Managed -> Detached(merge) / Removed
Lazy(collections default) vs Eager(@ManyToOne/@OneToOne default) -> prefer LAZY
N+1: 1 parent query + N child queries -> fetch join / @EntityGraph / @BatchSize / DTO
LazyInitializationException = lazy access after tx closed
Repos: Crud < PagingAndSorting < JpaRepository. Page(count) vs Slice(no count)
@Transactional = AOP proxy; rollback only unchecked (rollbackFor for checked); readOnly skips flush
Self-invocation bypasses proxy -> no tx
Propagation: REQUIRED default, REQUIRES_NEW suspends. Isolation: READ_COMMITTED typical
Cascade: PERSIST/MERGE/REMOVE/ALL; orphanRemoval deletes detached children
FK owned by @ManyToOne side; parent uses mappedBy. @Version = optimistic locking
```

Module 8 — Mock Interview (answer, then continue)

1. "Explain the N+1 problem with SQL, then show me three different fixes and their trade-offs."
2. "I loaded an entity, changed a field, never called save — it still updated. Why?"
3. "A checked exception was thrown mid-transaction but the DB kept the partial write. Why, and how do you fix it?"
4. "Why does accessing `order.getLines()` in the controller throw `LazyInitializationException`?"
5. "Design the JPA mapping for Order/OrderLine/Product with the right fetch and cascade settings, and justify each."

Continue to Module 9 when ready.

Module 9 — Spring Security

> Highest priority for any real backend role. JWT auth flow, the filter chain, and authentication vs authorization are asked constantly. Be able to *draw* the auth flow.

Node.js bridge: Like Passport.js middleware + bcrypt + jsonwebtoken, but as a structured **filter chain** in front of your controllers, with a shared `SecurityContext` per request.

9.1 Authentication vs Authorization

- **Authentication (AuthN)** — *who are you?* Verify identity (username/password, token, OAuth).
- **Authorization (AuthZ)** — *what can you do?* Check permissions/roles after identity is known.
- Order: authenticate first, then authorize. In Spring, AuthN populates the `SecurityContext`; AuthZ checks it (URL rules or `@PreAuthorize`).

9.2 The Security Filter Chain (core internal question)

Core Concept

Spring Security is a **chain of servlet filters** inserted before the `DispatcherServlet` (via a single `DelegatingFilterProxy` → `FilterChainProxy`). Each filter has one job; the request flows through until authenticated + authorized, then reaches your controller.

Internal Working — key filters in order

```
Request
  -> SecurityContextPersistenceFilter   (load SecurityContext, e.g., from session)
  -> [your JwtAuthenticationFilter]      (parse token, set Authentication)  <-- custom, stateless
  -> UsernamePasswordAuthenticationFilter (form login: build auth token)
  -> ... other filters (CORS, CSRF, logout) ...
  -> ExceptionTranslationFilter          (handle AuthN/AuthZ exceptions -> 401/403)
  -> AuthorizationFilter (FilterSecurityInterceptor pre-6) (enforce access rules)
  -> DispatcherServlet -> Controller
```

Authentication flow (form/password) — internal

```
1. UsernamePasswordAuthenticationFilter builds an (unauthenticated) Authentication token
2. -> AuthenticationManager (ProviderManager)
3. -> DaoAuthenticationProvider
        -> UserDetailsService.loadUserByUsername()  -> UserDetails (hash + authorities)
        -> PasswordEncoder.matches(raw, storedHash)  (BCrypt)
4. success -> authenticated Authentication stored in SecurityContextHolder
   failure -> AuthenticationException -> 401
```

Memory Diagram

```
[SecurityContextHolder] (ThreadLocal per request)
     holds -> Authentication { principal(UserDetails), authorities[ROLE_*], authenticated=true }
Controller / @PreAuthorize reads it to authorize.
```

Key components (memorize)

Component	Role
`AuthenticationManager` / `ProviderManager`	orchestrates authentication
`AuthenticationProvider` (`DaoAuthenticationProvider`)	does the actual verification
`UserDetailsService`	loads user (hash + roles) by username
`UserDetails`	user data + authorities
`PasswordEncoder` (`BCryptPasswordEncoder`)	hash + verify passwords
`SecurityContextHolder`	ThreadLocal holding the current `Authentication`
`SecurityFilterChain`	the ordered filters (config bean in Spring Security 6)

9.3 JWT Authentication (the most-asked security topic)

Core Concept

JWT (JSON Web Token) = a signed, self-contained token: **Header.Payload.Signature** (base64url). Stateless — the server verifies the **signature** instead of a session lookup, so any instance can validate it (great for microservices).

Structure & flow

```
JWT = base64(header).base64(payload).base64(HMAC/RSA signature)
  header : {"alg":"HS256","typ":"JWT"}
  payload: {"sub":"user123","roles":["ADMIN"],"exp":...,"iat":...}  (claims, NOT encrypted)
  signature: HMACSHA256(header.payload, secret)   <-- verifies integrity + authenticity

Login:  POST /login {user,pass} -> verify -> return signed JWT (+ refresh token)
Call:   GET /api/... Authorization: Bearer <jwt>
        JwtFilter: verify signature + exp -> build Authentication -> SecurityContext
```

Internal Working — stateless request

```
1. Client sends Authorization: Bearer <token>
2. Custom JwtAuthenticationFilter extracts + validates token (signature, expiry, issuer)
3. Builds an authenticated Authentication (username + authorities from claims)
4. Sets it in SecurityContextHolder -> downstream authorization works
5. No server session -> horizontally scalable
```

Best Answer

> “On login I verify credentials with BCrypt and issue a signed JWT whose payload carries the subject and roles. On each request a custom filter early in the chain validates the signature and expiry, rebuilds an `Authentication`, and puts it in the `SecurityContextHolder` — so auth is stateless and any service instance can verify it without a shared session store. The payload is signed, not encrypted, so I never put secrets in it, keep access tokens short-lived, and use refresh tokens for renewal.”

JWT traps (interviewers probe these)

- **Payload is not encrypted** — anyone can base64-decode claims. Never store secrets.
- **Can’t revoke easily** — stateless means you need short expiry + a refresh/blacklist strategy for logout.
- Always validate `alg` (reject `none`), expiry, issuer/audience.
- Store carefully on the client (httpOnly cookie vs localStorage → XSS/CSRF trade-offs).

Coding Example (Spring Security 6 style)

```
@Configuration
@EnableWebSecurity
class SecurityConfig {
    @Bean
    SecurityFilterChain chain(HttpSecurity http, JwtAuthFilter jwtFilter) throws Exception {
        http
          .csrf(csrf -> csrf.disable())                       // stateless API -> CSRF off
          .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
          .authorizeHttpRequests(auth -> auth
              .requestMatchers("/api/auth/**").permitAll()
              .requestMatchers("/api/admin/**").hasRole("ADMIN")
              .anyRequest().authenticated())
          .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }

    @Bean PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

    @Bean
    AuthenticationManager authManager(AuthenticationConfiguration c) throws Exception {
        return c.getAuthenticationManager();
    }
}
```

9.4 BCrypt & PasswordEncoder

- **Never store plaintext or fast hashes (MD5/SHA-1)** for passwords. Use **BCrypt** (or Argon2/PBKDF2): a slow, salted, adaptive hash.
- BCrypt embeds a random **salt** and a **work factor** (strength, default 10) in the hash → same password yields different hashes; brute-force is deliberately slow.
- Verify with `encoder.matches(raw, storedHash)` — never decrypt (it's one-way).

9.5 CORS & CSRF (always confused — nail the difference)

	CORS	CSRF
Stands for	Cross-Origin Resource Sharing	Cross-Site Request Forgery
What	Browser policy: which origins may call your API	Attack: forged request using victim's cookies
Direction	Relaxes same-origin policy	Protects state-changing requests
Config	allowed origins/methods/headers	token per form/request
For JWT APIs	configure allowed origins	usually **disabled** (no cookies → no CSRF)

- **CORS** is not security — it's the browser deciding if JS from origin B may read responses from your API. Configure allowed origins/methods.
- **CSRF** matters for **cookie/session** auth (browser auto-sends cookies). With **stateless JWT in an Authorization header**, CSRF protection is typically disabled because there's no ambient cookie to forge.

Module 9 — Top 25 Interview Questions (senior answers)

1. **AuthN vs AuthZ?** Who you are vs what you can do.
2. **How does Spring Security work?** Ordered servlet filter chain before controllers.

3. **Key filter-chain components?** AuthenticationManager, provider, UserDetailsService, PasswordEncoder, SecurityContextHolder.
4. **Form-login auth flow?** Filter → ProviderManager → DaoProvider → UserDetailsService + encoder → SecurityContext.
5. **What is SecurityContextHolder?** ThreadLocal holding the current Authentication.
6. **What is UserDetailsService?** Loads user (hash + authorities) by username.
7. **What does AuthenticationProvider do?** Performs actual credential verification.
8. **JWT structure?** header.payload.signature (base64url).
9. **Is JWT encrypted?** No — signed; payload is readable. Don't store secrets.
10. **How is a JWT validated per request?** Custom filter verifies signature+expiry, sets Authentication.
11. **Why is JWT good for microservices?** Stateless; any instance verifies via signature.
12. **JWT revocation/logout problem?** Stateless → short expiry + refresh/blacklist.
13. **Access vs refresh token?** Short-lived API token vs long-lived renewal token.
14. **Why BCrypt over MD5/SHA?** Slow, salted, adaptive work factor.
15. **Does BCrypt use a salt?** Yes — embedded, random per hash.
16. **How to verify a password?** `encoder.matches(raw, hash)` (one-way).
17. **CORS vs CSRF?** Browser origin policy vs forged-request attack.
18. **Why disable CSRF for JWT APIs?** No cookies → nothing to forge.
19. **When keep CSRF on?** Cookie/session-based auth.
20. **Method vs URL security?** `@PreAuthorize`/`@Secured` vs `authorizeHttpRequests`.
21. **Roles vs authorities?** Role = `ROLE_`-prefixed authority; authorities are granular.
22. **How to make it stateless?** `SessionCreationPolicy.STATELESS`.
23. **401 vs 403?** Unauthenticated vs authenticated-but-forbidden.
24. **How does @PreAuthorize work?** AOP proxy evaluating SpEL against the Authentication.
25. **OAuth2/OIDC vs JWT?** Delegated auth protocol vs token format (often used together).

Module 9 — Top Coding Questions

- Implement a `JwtAuthenticationFilter` (extract, validate, set context).
- Configure a stateless `SecurityFilterChain` with role-based URL rules.
- Implement `UserDetailsService` backed by a JPA repository + BCrypt.
- Add `@PreAuthorize("hasRole('ADMIN')")` to a method and test 403.
- Build login endpoint issuing access + refresh tokens.

Module 9 — Common Follow-ups

- "How do you log out / revoke a JWT?" (short expiry, refresh rotation, denylist.)
- "Where do you store the token client-side and why?" (httpOnly cookie vs localStorage trade-offs.)
- "Someone base64-decodes your JWT and reads the roles — is that a vulnerability?" (No — it's signed, not secret; don't put secrets in it.)

Module 9 — One-Page Cheat Sheet

```
AuthN(who) then AuthZ(what). Filter chain before DispatcherServlet.
Flow: Filter -> AuthenticationManager(ProviderManager) -> DaoAuthenticationProvider
      -> UserDetailsService + PasswordEncoder(BCrypt) -> SecurityContextHolder(ThreadLocal)
JWT = header.payload.signature (signed, NOT encrypted). Verify signature+exp per request in a filter.
Stateless -> scalable; revoke via short expiry + refresh token. Never put secrets in payload.
BCrypt = slow, salted, adaptive; matches(raw,hash), one-way.
CORS = browser origin policy (not security). CSRF = forged cookie request; disable for JWT header APIs.
Config(SS6): SecurityFilterChain bean, STATELESS, authorizeHttpRequests, addFilterBefore(jwt,...)
401=unauthenticated, 403=forbidden. @PreAuthorize = method security via AOP.
```

Module 9 — Mock Interview (answer, then continue)

1. “Draw and explain the JWT authentication flow from login to an authorized API call.”
2. “Walk through the Spring Security filter chain and where your custom JWT filter goes.”
3. “Why do we disable CSRF for a JWT API but not for a session-cookie app?”
4. “Why BCrypt and not SHA-256 for passwords? What does the work factor do?”
5. “How do you handle logout/token revocation in a stateless JWT system?”

Continue to Module 10 when ready.

Module 10 — REST API Design

> High priority. "Design a REST API for X", idempotency, and correct status codes come up in almost every backend interview. You already know HTTP from Express — this formalizes it.

Node.js bridge: Same HTTP semantics you used in Express; the difference is Java interviewers expect precise vocabulary (idempotency, HATEOAS awareness, versioning strategy).

10.1 REST Principles

REST = **RE**presentational **S**tate **T**ransfer. Constraints worth naming: - **Client-server** + **stateless** — no server session; each request carries all context (auth token, params). Enables horizontal scaling. - **Uniform interface** — resources identified by URIs (`/users/42`), manipulated via representations (JSON), standard verbs. - **Cacheable** — responses declare cacheability (`Cache-Control`, `ETag`). - **Layered system** — client can't tell if it talks to the server or a proxy/gateway. - **Resource-oriented naming**: nouns not verbs, plural collections — `GET /users/42/orders`, **not** `/getUserOrders`.

> **Trap:** "REST is just HTTP + JSON." Interviewers want *statelessness*, *resource nouns*, and *correct verb/status usage*.

10.2 HTTP Methods, Idempotency & Safety (the key table)

Method	Purpose	Safe (no change)	Idempotent	Body
GET	read	Yes	Yes	no
POST	create / non-idempotent action	No	**No**	yes
PUT	full replace/create-at-id	No	**Yes**	yes
PATCH	partial update	No	usually No*	yes
DELETE	remove	No	**Yes**	maybe

* PATCH *can* be idempotent depending on the patch semantics.

Idempotency (must-explain concept)

An operation is **idempotent** if making it **N times has the same effect as once**. - `PUT /users/42 {full body}` — repeat → same final state → idempotent. - `POST /users` — repeat → creates duplicates → **not** idempotent. - **DELETE** — deleting twice: resource still gone (often 404 the second time, but state is idempotent).

Production pattern — idempotency keys: For `POST /payments`, clients send an `Idempotency-Key` header; the server stores the key + result so a retried request returns the original response instead of double-charging. Classic Stripe interview point.

Best Answer

> "GET is safe and idempotent; PUT and DELETE are idempotent but not safe; POST is neither, so retries can duplicate. For non-idempotent POSTs like payments I use an idempotency key the server dedupes on, so client retries after a timeout don't double-charge."

10.3 HTTP Status Codes (know the exact ones)

Range	Meaning	Common codes
2xx success		200 OK, 201 Created (+`Location`), 202 Accepted (async), 204 No Content (delete)
3xx redirect		301 Moved Permanently, 304 Not Modified (caching/ETag)
4xx client error		400 Bad Request, 401 Unauthorized (authN), 403 Forbidden (authZ), 404 Not Found, 405 Method Not Allowed, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests
5xx server error		500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

Traps: 401 vs 403 (not authenticated vs authenticated-but-forbidden); return **201 + Location** on create, **204** on delete with no body; **409** for conflicts (duplicate), **422** for semantic validation failure, **429** for rate limiting.

10.4 Pagination, Filtering, Sorting

- **Offset pagination:** `GET /users?page=2&size=20&sort=name,asc`. Simple; slow/inconsistent for deep pages on large tables. In Spring Data: `Pageable` → `Page<T>` (includes total count).
- **Cursor/keyset pagination:** `?after=<lastId>&size=20`. Stable and fast for large datasets (used by Stripe/Slack). Preferred at scale.
- **Filtering:** query params `?status=ACTIVE&minAge=18` (or a query DSL / Specifications).
- **Sorting:** `?sort=field,dir`.
- Return metadata: total count / next-cursor, and consider envelope vs headers (`Link` header).

```
@GetMapping("/users")
Page<UserDto> list(@RequestParam(defaultValue="0") int page,
                   @RequestParam(defaultValue="20") int size,
                   @RequestParam(required=false) Status status,
                   @SortDefault(sort="createdAt", direction=Sort.Direction.DESC) Pageable pageable) {
    return service.search(status, pageable);
}
```

10.5 API Versioning

Strategy	Example	Notes
URI path (most common)	`/api/v1/users`	simplest, visible, cache-friendly
Header	`Accept: application/vnd.app.v2+json`	clean URLs, harder to test
Query param	`/users?version=2`	easy but muddies caching

Best answer: “URI versioning (`/v1`) is the pragmatic default — explicit and easy to route at the gateway. I version only on breaking changes, keep backward compatibility via additive changes, and deprecate old versions with a sunset policy.”

10.6 Good REST API design (senior checklist)

- Nouns + plurals, nested for relationships: `/customers/1/orders`.
- Consistent error envelope: `{ "code", "message", "traceId", "fieldErrors" }`.

- Validate input (`@Valid`) → 400/422; centralize with `@RestControllerAdvice`.
- Use DTOs, never expose JPA entities directly (avoids over-exposing fields + lazy issues).
- `ResponseEntity` for explicit status/headers; `201 + Location` on create.
- Document with OpenAPI/Swagger.
- Rate limit (429), paginate everything that can grow, support ETags for caching.

Module 10 — Top 25 Interview Questions (senior answers)

1. **What is REST?** Stateless, resource-oriented architecture over HTTP with a uniform interface.
2. **Is REST stateless? Why does it matter?** Yes — each request self-contained → horizontal scale.
3. **PUT vs POST?** Idempotent replace/create-at-id vs non-idempotent create.
4. **PUT vs PATCH?** Full replace vs partial update.
5. **What is idempotency?** N calls = 1 call's effect; GET/PUT/DELETE yes, POST no.
6. **How to make POST safe to retry?** Idempotency key dedup on the server.
7. **Which methods are safe?** GET (and HEAD/OPTIONS) — no state change.
8. **201 vs 200 vs 204?** Created (+Location) / OK / No Content.
9. **401 vs 403?** Unauthenticated vs forbidden.
10. **409 vs 422?** Conflict (duplicate/state) vs semantic validation failure.
11. **429?** Too Many Requests (rate limiting).
12. **502 vs 503 vs 504?** Bad gateway / unavailable / gateway timeout.
13. **Offset vs cursor pagination?** Simple vs stable-and-fast at scale.
14. **How to paginate in Spring Data?** `Pageable` → `Page`/`Slice`.
15. **How do you version APIs?** URI path (default), header, or query.
16. **When to bump the version?** Only breaking changes; keep additive changes compatible.
17. **Why DTOs over entities?** Decoupling, security, avoid lazy/serialization issues.
18. **How to design error responses?** Consistent envelope with code/message/traceId.
19. **What is HATEOAS?** Hypermedia links in responses (aware-of; rarely fully implemented).
20. **Idempotent DELETE returning 404 twice — ok?** Yes; final state is idempotent.
21. **ETag / 304?** Conditional caching to save bandwidth.
22. **Path vs query params?** Identify resource vs filter/paginate/sort.
23. **Bulk operations design?** Batch endpoint + partial success (207-style) semantics.
24. **How to secure a REST API?** AuthN (JWT/OAuth), AuthZ, HTTPS, rate limit, input validation.
25. **REST vs gRPC vs GraphQL?** Simple/ubiquitous vs high-perf RPC vs flexible client-driven queries.

Module 10 — Top Coding Questions

- Design + implement CRUD for `/api/v1/orders` with correct verbs/status codes.
- Add pagination, filtering, and sorting to a list endpoint.
- Implement idempotent payment creation with an `Idempotency-Key`.
- Build a consistent error response with `@RestControllerAdvice`.

Module 10 — Common Follow-ups

- "A client's payment request times out and it retries — how do you prevent a double charge?"
- "You added a required field to the response — is that a breaking change? Version?"
- "Why not return the JPA entity directly?"

Module 10 — One-Page Cheat Sheet

```
REST: stateless, resource nouns (plural), uniform interface, cacheable
GET safe+idempotent | PUT/DELETE idempotent | POST neither | PATCH partial
Idempotency: N=1 effect. POST payments -> Idempotency-Key dedup (no double charge)
Status: 200 OK | 201 Created(+Location) | 204 No Content | 400 bad | 401 authN | 403 authZ |
        404 | 409 conflict | 422 validation | 429 rate limit | 500 | 502/503/504 gateway
Pagination: offset(page/size, Page) vs cursor(after=lastId, scalable)
Versioning: /api/v1 (default) | Accept header | query. Bump only on breaking change.
Use DTOs not entities; @Valid -> 400; @RestControllerAdvice error envelope; ResponseEntity
```

Module 10 — Mock Interview (answer, then continue)

1. “Design the REST API for an e-commerce order service — resources, verbs, status codes, pagination.”
2. “Explain idempotency and how you’d make `POST /payments` safe under client retries.”
3. “Give me the right status code for: create success, validation failure, duplicate, unauthorized, rate-limited.”
4. “Offset vs cursor pagination — which for a 100M-row table and why?”
5. “How do you version an API and decide when a change is breaking?”

Continue to Module 11 when ready.

Module 11 — Microservices (Interview Focus)

> High priority for “Java Microservices Developer” roles. You need interview-relevant depth on the Spring Cloud stack: gateway, discovery, config, Feign, resilience, and messaging.

Node.js bridge: Same distributed-systems ideas you know from AWS (API Gateway, service discovery, SQS/SNS, circuit breakers). Here they’re the Spring Cloud + Resilience4j + Kafka ecosystem.

11.1 Monolith vs Microservices

	Monolith	Microservices
Deploy	one unit	many independent services
Scaling	whole app	per-service
Data	one shared DB	DB-per-service
Coupling	tight	loose (network boundaries)
Failure blast radius	whole app	isolated (if resilient)
Complexity	low ops	high ops (network, observability, consistency)

Best answer: “Microservices give independent deployability, per-service scaling, and tech flexibility, at the cost of distributed-systems complexity — network failures, eventual consistency, and observability. I’d start with a well-structured monolith and split out services along clear bounded contexts only when scaling or team autonomy demands it.”

Traps: proposing microservices for a small app; sharing one database across services (defeats the point — each service owns its data).

11.2 API Gateway

- Single entry point for all clients; routes to backend services. Handles cross-cutting concerns: **auth, rate limiting, CORS, request routing, load balancing, aggregation**.
- **Spring Cloud Gateway** (reactive, WebFlux-based; replaced Zuul). Define route predicates + filters.

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://ORDER-SERVICE          # lb:// = load-balanced via discovery
          predicates: [ Path=/api/orders/** ]
          filters: [ StripPrefix=2 ]
```

- Benefits: clients don’t know internal topology; centralized security/observability.

11.3 Service Discovery — Eureka

- Services **register** themselves with a registry (**Eureka Server**) and **discover** each other by logical name instead of hardcoded host:port — essential when instances scale up/down dynamically.

- **Internal working:** each instance registers (name + host:port), sends **heartbeats**; Eureka evicts instances that stop heart-beating. Clients cache the registry and load-balance across instances (client-side LB).
- `@EnableEurekaServer` (registry) / `@EnableDiscoveryClient` (services).

```
[order-svc instances] --register/heartbeat--> [Eureka Server] <--fetch registry-- [gateway/clients]
call: use "ORDER-SERVICE" logical name -> resolve to a live instance -> load-balance
```

11.4 Config Server

- **Spring Cloud Config Server** centralizes configuration (backed by a Git repo) for all services and environments; supports refresh without redeploy (`@RefreshScope` + `/actuator/refresh` or Spring Cloud Bus).
- Benefit: one place for config, versioned in Git, per-profile (`service-prod.yml`), secrets via Vault integration.

11.5 Feign Client & Load Balancing

Feign (declarative REST client)

Define an interface; Spring Cloud generates the HTTP client — no boilerplate `RestTemplate`/`WebClient` code.

```
@FeignClient(name = "ORDER-SERVICE")           // resolves via discovery + load balancer
interface OrderClient {
    @GetMapping("/api/orders/{id}")
    OrderDto getOrder(@PathVariable Long id);
}
```

Load Balancing

- **Client-side LB** (Spring Cloud LoadBalancer, replaced Ribbon): the caller picks an instance from the discovery registry (round-robin by default). `lb://SERVICE-NAME`.
- **Server-side LB** (gateway / cloud LB): a central component distributes requests.

11.6 Circuit Breaker & Resilience4j (key resilience topic)

Why

In a distributed system, one slow/failing downstream can cascade and exhaust threads across the whole system. Resilience patterns contain the failure.

Circuit Breaker states

```
CLOSED  --(failure rate > threshold)-->  OPEN  --(after wait)-->  HALF_OPEN
  ^                                                                  |
  +--------------(trial calls succeed)-------------------------------+
CLOSED     : calls pass through, failures counted
OPEN       : calls fail fast immediately (no downstream call) -> fallback
HALF_OPEN  : allow a few trial calls; success -> CLOSED, failure -> OPEN
```

Resilience4j patterns (lightweight, replaced Hystrix)

- **Circuit Breaker** — stop calling a failing service; fail fast + fallback.
- **Retry** — retry transient failures (with backoff; only for idempotent ops!).
- **Rate Limiter** — cap request rate.
- **Bulkhead** — isolate resources/thread pools so one dependency can't starve others.
- **TimeLimiter** — bound call duration.

```
@CircuitBreaker(name = "orderService", fallbackMethod = "fallback")
@Retry(name = "orderService")
public OrderDto getOrder(Long id) { return orderClient.getOrder(id); }

private OrderDto fallback(Long id, Throwable t) {
    return OrderDto.unavailable(id);            // graceful degradation
}
```

Best Answer

> "A circuit breaker prevents cascading failures: after the failure rate crosses a threshold it trips OPEN and fails fast with a fallback instead of hammering a dead service, then probes in HALF_OPEN before closing. I pair it with retries (only for idempotent calls, with backoff), timeouts, and bulkheads via Resilience4j so a single slow dependency can't exhaust the whole service's threads."

11.7 Distributed Transactions & the Saga Pattern (high level)

The problem

With a **database per service**, you can't use a single ACID transaction across services. Two-phase commit (2PC) is slow and doesn't scale. Solution: **eventual consistency** via Sagas.

Saga Pattern

A distributed transaction = a sequence of **local transactions**, each publishing an event that triggers the next. On failure, run **compensating transactions** to undo prior steps.

```
Order Saga (choreography):
 OrderCreated -> [Payment] PaymentDone -> [Inventory] Reserved -> [Shipping] Shipped
 If Inventory fails -> emit compensation -> Payment refunded, Order cancelled
```

- **Choreography** — services react to events (decentralized, simple, harder to trace).
- **Orchestration** — a central orchestrator directs steps (clearer, single coordinator).
- Also know **Outbox pattern** (atomic DB write + event) and **idempotent consumers**.

11.8 Kafka & RabbitMQ Basics

Kafka (distributed event log)

- **Topic** split into **partitions** (ordering + parallelism per partition); messages retained (replayable log).
- **Producer** writes; **consumers** in a **consumer group** share partitions (each partition to one consumer in the group → scales horizontally).
- **Offset** tracks read position (consumer-committed). High throughput, durable, replayable.
- Use for: event streaming, high-volume pipelines, event sourcing, decoupling.

RabbitMQ (message broker / queue)

- **Exchange** → **binding** → **queue** routing (direct/topic/fanout). Push-based, per-message ack, smart broker.
- Use for: task queues, RPC, complex routing, lower-volume reliable messaging.

Kafka vs RabbitMQ (guaranteed comparison)

	Kafka	RabbitMQ
Model	distributed log (pull)	queue/broker (push)
Retention	retained, replayable	removed after ack
Throughput	very high	high
Ordering	per partition	per queue
Best for	event streaming, analytics	task queues, routing

Best Answer

> “Kafka is a partitioned, replayable log — great for high-throughput event streaming and event sourcing, with ordering per partition and consumer groups for horizontal scaling. RabbitMQ is a smart broker with flexible exchange routing and per-message acks — great for task queues and RPC. I choose Kafka for streaming/high volume and RabbitMQ for complex routing and work queues. For cross-service consistency I use async events with idempotent consumers and the outbox pattern rather than distributed transactions.”

Module 11 — Top 25 Interview Questions (senior answers)

1. **Monolith vs microservices trade-offs?** Independent deploy/scale vs distributed complexity.
2. **When NOT to use microservices?** Small app/team; start monolith, split on need.
3. **Why DB-per-service?** Loose coupling, independent scaling/schema; no shared DB.
4. **What is an API Gateway?** Single entry: routing, auth, rate limit, aggregation.
5. **Spring Cloud Gateway vs Zuul?** Reactive/current vs legacy blocking.
6. **What is service discovery / Eureka?** Dynamic registry; register + heartbeat + discover by name.
7. **Client vs server-side load balancing?** Caller picks instance vs central LB.
8. **What is Feign?** Declarative REST client interface; integrates with discovery + LB.
9. **What is Config Server?** Centralized, Git-backed config across services/envs.
10. **@RefreshScope?** Reload config without redeploy.
11. **Circuit breaker states?** CLOSED → OPEN → HALF_OPEN.
12. **Why circuit breaker?** Prevent cascading failures; fail fast + fallback.
13. **Resilience4j patterns?** CircuitBreaker, Retry, RateLimiter, Bulkhead, TimeLimiter.
14. **Resilience4j vs Hystrix?** Lightweight, functional, active vs deprecated.
15. **When to retry?** Transient failures + idempotent operations, with backoff.
16. **What is a bulkhead?** Resource isolation so one dependency can’t starve others.
17. **Distributed transaction problem?** No cross-service ACID; 2PC doesn’t scale.
18. **Saga pattern?** Local txns + events + compensating txns for rollback.
19. **Choreography vs orchestration?** Event-reactive decentralized vs central coordinator.
20. **Outbox pattern?** Atomic DB write + event to avoid dual-write inconsistency.
21. **Kafka topic/partition/offset?** Log split for parallelism; consumer-tracked read position.
22. **Consumer group?** Parallel consumption; one partition per consumer in group.
23. **Kafka vs RabbitMQ?** Replayable log/streaming vs broker/queue routing.
24. **How to trace across services?** Correlation id + distributed tracing (Sleuth/Micrometer + Zipkin/OTel).
25. **How to ensure idempotent consumers?** Dedup by message/event id.

Module 11 — Top Coding / Design Questions

- Set up a Feign client with a Resilience4j circuit breaker + fallback.
- Configure a Spring Cloud Gateway route with auth + rate-limit filters.
- Design an order-checkout saga with compensations.
- Design a Kafka producer/consumer for an order-events topic (partitioning + consumer group).
- Add distributed tracing/correlation ids across two services.

Module 11 — Common Follow-ups

- “A downstream service is slow — how do you keep it from taking down your service?” (timeout + circuit breaker + bulkhead.)
- “How do you keep two services’ data consistent without 2PC?” (saga + outbox + idempotency.)
- “Why partition a Kafka topic?” (parallelism + ordering within a key.)

Module 11 — One-Page Cheat Sheet

```
Monolith(simple) vs Microservices(independent deploy/scale, DB-per-service, distributed complexity)
API Gateway: single entry, routing/auth/rate-limit (Spring Cloud Gateway, reactive)
Eureka: register + heartbeat + discover by name; client-side LB (Spring Cloud LoadBalancer)
Feign: declarative REST client (@FeignClient), lb://SERVICE
Config Server: Git-backed central config; @RefreshScope
Resilience4j: CircuitBreaker(CLOSED->OPEN->HALF_OPEN + fallback), Retry(idempotent+backoff),
  RateLimiter, Bulkhead(isolation), TimeLimiter
Distributed tx: no 2PC -> Saga(local txns + compensations), choreography vs orchestration, Outbox,
idempotent consumers
Kafka: partitioned replayable log, consumer groups, offsets -> streaming/high volume
RabbitMQ: exchange->queue routing, per-msg ack -> task queues/routing
Tracing: correlation id + Micrometer/OTel + Zipkin
```

Module 11 — Mock Interview (final module — answer these)

1. “A downstream inventory service starts timing out under load. Walk me through every resilience mechanism you’d apply and why.”
2. “Design order checkout across order, payment, inventory, and shipping services without distributed transactions.”
3. “Explain the circuit breaker state machine and how you’d tune the thresholds.”
4. “Kafka vs RabbitMQ — pick one for an order-events pipeline and justify it.”
5. “How does a request flow from the client through the gateway to a specific service instance?”

Bootcamp Wrap-Up — 5-Day Revision Plan

- **Day 1:** Modules 1–2 (Core Java + Collections). Re-read every ASCII diagram + cheat sheet.
- **Day 2:** Modules 3–5 (Java 8+, Exceptions, Concurrency). Do the coding questions.
- **Day 3:** Modules 6–7 (Spring + Boot). Be able to trace a request end-to-end.
- **Day 4:** Modules 8–9 (JPA + Security). Draw the N+1 fix and the JWT flow from memory.
- **Day 5:** Modules 10–11 (REST + Microservices) + all mock interviews out loud.

Interview-day rules: state the *why*, name the internal mechanism, mention one trap, then give the senior best-answer. Relate to your Node/AWS experience where natural. You’ve got this.